Fashion E-Commerce with Auction System

A project work submitted to Madurai Kamaraj University in

partial fulfillment for the award of the degree Master of

Computer Applications

Project work undertaken at

DCE Technology , Virudhunagar,India

Under the Guidance of

Mrs. P. MURUGESWARI. M.C.A., M.Phil., SET.,

Assistant Professor

By

SHEIK ABDULLA S
24SPCA043

Department of Computer Applications

Virudhunagar Hindu Nadars' Senthikumara Nadar College

An Autonomous Institution, Affiliated to Madurai Kamaraj University

Accredited with 'A+' Grade (5th Cycle)

Virudhunagar
April 2026

Virudhunagar Hindu Nadars' Senthikumara Nadar College

An Autonomous Institution, Affiliated to Madurai Kamaraj University

Accredited with 'A+' Grade (5th Cycle) by NAAC

Virudhunagar

Department of Computer Applications

Certificate

This is to certify that the project work entitled "**Fashion E-Commerce with Auction System**" is submitted by **SHEIK ABDULLA S**, Register No. **24SPCA043**at **Department of Computer Applications** in partial fulfillment of the requirements for the award of **Master of Computer Applications** degree by the Madurai Kamaraj University during the academic years 2024-2026. The project represents the independent work done by the candidate under my guidance.

Internal Guide **Head of the Department**

Submitted for the Viva-Voce examination held on **13.04.2026** at Virudhunagar Hindu Nadars' Senthikumara Nadar College (Autonomous), Virudhunagar.

Internal Examiner **External Examiner**

Virudhunagar Hindu Nadars' Senthikumara Nadar College

An Autonomous Institution, Affiliated to Madurai Kamaraj University

Accredited with 'A+' Grade (5th Cycle) by NAAC

Virudhunagar

Department of Computer Applications

DECLARATION

I hereby declare that the project report entitled "Fashion E-Commerce with Auction System" submitted to Madurai Kamaraj University in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications is a record of original work carried out by me under the supervision of my guide. I further declare that this report has not been submitted in part or in full to any other university or institution for the award of any degree or diploma.

Place: _____

Date: _____

Signature of the Candidate: _____

ACKNOWLEDGEMENT

I thank GOD the Almighty who showered his blessings on me for the successful completion of this project work.

I am very much grateful to out principal **Dr. A. Sarathi. M.Sc., M.Phil., Ph.D.,** V.H.N.S.N. College Virudhunagar who provide me a well infrastructure for doing this work.

I am very much grateful to out SF Co-Ordinator **Dr. A. Kalidass. M.A., M.Phil., M.Ed., Ph.D.,** V.H.N.S.N. College, Virudhunagar provide me a well infrastructure for doing this work.

I Express my sincere thanks to out Head of the Department **Mr. D. Rajkumar. M.C.A., M.Phil.,** for giving me permission to do this project work.

I have pleasure in thanking **Mrs. P. MURUGESWARI. M.C.A., M.Phil., SET.,** Department of Computer Applications for providing me an excellent guidance and for giving suggestions to do this project work.

I extend my heartfelt thanks to **Mr. S. KARTHIKEYAN. CEO** of DCE Technology, for his outstanding guidance, continuous support, and valuable suggestions in successfully completing this project.

I express my sincere thanks to **Mr. S. STHIYAMOORTHI.** Team Lead, DCE TECHNOLOGY for Mentoring me an excellence and for giving suggestions to do this project work.

I would like to thank all out teaching and non-teaching staffs of out Department of Computer Applications V.H.N.S.N. College, Virudhunagar for their kindness and for their

help. Last but no least, I express my sincere thanks to my parents, friends and have helped to make this project a successful one.

I also thank my friends, classmates, and family members for their moral support and cooperation during the preparation of this report. Finally, I express my gratitude to everyone who directly or indirectly contributed to the successful completion of the project titled "Fashion E-Commerce with Auction System".

Virudhunagar Hindu Nadars' Senthikumara Nadar College

An Autonomous Institution, Affiliated to Madurai Kamaraj University

Accredited with 'A+' Grade (5th Cycle) by NAAC

Virudhunagar

Department of Computer Applications

DECLARATION

I, **SHEIK ABDULLA S,** Register No. **24SPCA043**, hereby declare that the project work entitled "**Fashion E-Commerce with Auction System**" submitted to Madurai Kamaraj University, Madurai, in partial fulfillment of requirements for the award of the degree of Master of Computer Applications of record of my individual work during my period of study in Virudhunagar Hindu Nadars' Senthikumara Nadar College (Autonomous), Virudhunagar under the guidance **Mrs. P. MURUGESWARI. M.C.A., M.Phil., SET.,**

Date : 13.04.2026

Place: Virudhunagar

Signature of the student,

SHEIK ABDULLA S

ABSTRACT

The project entitled "Fashion E-Commerce with Auction System" is a web-based platform developed to unify conventional online shopping with real-time auction management. The application is designed for the fashion retail domain and supports the core activities of product browsing, customer registration, cart management, wallet operations, order processing, digital payment, vendor administration, and live auction participation. By integrating an auction engine into an e-commerce environment, the system creates a flexible commercial model in which regular products can be sold directly while selected premium items can be offered through competitive bidding.

The proposed solution is implemented using the MERN stack, namely MongoDB, Express.js, React.js, and Node.js. The backend exposes modular REST APIs and real-time auction events through Socket.IO, while the frontend provides a role-based user interface for administrators, customers, and vendors. The database design stores products, auctions, bids, winners, payments, carts, orders, notifications, and wallet transactions in a structured and scalable manner.

This project addresses the limitations of traditional e-commerce systems, where price is static and user engagement is limited. The auction module improves interactivity, increases perceived value for premium products, and enables dynamic price discovery. The system is therefore suitable for fashion businesses that wish to combine product merchandising with live digital selling experiences.

TABLE OF CONTENT

1. INTRODUCTION

1.1 ORGANISATION PROFILE

For the purpose of this academic project, the organisation considered is **DCE Technology**, located in Virudhunagar. It is a software development company that focuses on building custom software solutions, web applications, mobile applications, and cloud-based systems for various business needs.

The company is known for delivering reliable, scalable, and user-friendly applications using modern technologies and agile development practices. DCE Technology emphasizes quality design, efficient system architecture, and enhanced user experience in all its projects.

The organisation also encourages innovation by implementing advanced features such as real-time systems, dynamic user interaction, and modern digital commerce solutions. This makes it a suitable environment for developing complex applications like a fashion e-commerce platform with auction-based functionality.

1.2 ABSTRACT

Traditional fashion e-commerce platforms mainly operate using a fixed-price model, where customers browse products, add them to the cart, and complete purchases. While this approach is effective, it does not fully support the sale of premium or limited-edition fashion items that generate high demand and competition.

This project proposes an enhanced fashion e-commerce system that integrates an auction-based selling mechanism along with the standard shopping model. The system allows exclusive products to be sold through real-time bidding, while regular products continue to be available at fixed prices.

The platform provides a dual-mode functionality where administrators can create and manage auction rooms, assign products, monitor live bids, and declare winners. Customers can register, maintain a digital wallet, participate in auctions, place bids in real time, and also use standard features such as browsing products, adding to cart, making payments, and tracking orders.

By combining auction and traditional e-commerce features, the system improves user engagement, increases business opportunities, and provides a flexible and dynamic shopping experience.

1.3 PROBLEM DEFINITION

Traditional fashion e-commerce systems face several operational and experiential limitations. Product prices are normally static, customer interaction is limited to browsing and buying, and premium goods often fail to achieve optimal market value because there is no transparent mechanism for competitive price discovery. In addition, separate auction portals create fragmentation in user identity, payment flow, and inventory control.

The problem addressed in this project is the absence of a unified platform that can support both fixed-price fashion shopping and controlled live auctions using a common customer account, shared product database, wallet validation, and role-based administration. The system therefore aims to provide secure participation, real-time updates, proper winner declaration, and seamless movement between shopping and bidding processes.

2. SYSTEM ANALYSIS

2.1 EXISTING SYSTEM ARCHITECTURE

In an ordinary e-commerce application, products are listed with fixed prices and customers purchase items directly through the cart and payment workflow. While this approach is sufficient for common retail transactions, it is less effective for scarce, premium, or trend-sensitive fashion products. The existing approach does not encourage competitive engagement, and it cannot adjust product value according to real-time demand.

Another limitation of the existing system is that customer excitement and platform stickiness remain comparatively low. Users visit the platform, complete the purchase, and leave without any live participation model. There is also no native support for auction room creation, bidding validation, winner announcement, or synchronized bid updates.

• Static pricing does not support premium-value discovery.

• No real-time bidding or auction room management.

• Low customer engagement for exclusive fashion products.

• Separate systems are often required for direct sales and auctions.

• Administrative control over live selling events is limited.

2.2 PROPOSED SYSTEM ARCHITECTURE

The proposed system integrates auction functionality into a full-fledged fashion e-commerce platform. It supports three major user roles, namely administrator, vendor, and customer. Vendors can manage products, customers can shop and participate in auctions, and administrators can create rooms, assign products, manage live auction flow, and supervise winner selection. The application therefore maintains one digital ecosystem for both ordinary purchasing and event-driven product bidding.

The backend uses modular APIs and real-time socket communication to keep auction state synchronized among participants. Wallet balance checking and role validation improve the reliability of bidding. Since orders, payments, products, and auction winners are stored in one database environment, the system reduces duplication and improves consistency.

• Unified platform for fixed-price shopping and live auctions.

• Real-time bidding with room-based synchronization.

• Wallet-driven participation and winner settlement tracking.

• Role-based dashboards for admin, customer, and vendor.

• Integrated modules for catalog, cart, payment, order, and auction.

2.3 USER INTERFACE REQUIREMENTS

The user interface is required to be responsive, role-based, simple to navigate, and visually appropriate for a fashion commerce environment. It must provide structured dashboards, readable product cards, secure forms for login and registration, and real-time live bidding components that display the current product, current price, next bid amount, timer, participant status, and winner details.

• Login, registration, and profile management pages for all users.

• Product browsing, category filtering, product detail, cart, and checkout pages.

• Admin dashboards for customers, vendors, products, orders, and auctions.

• Auction room screens with timer, bid panel, queue, live bids, and sold items.

• Notification and wallet views for transaction transparency.

Home Page and Product Catalog

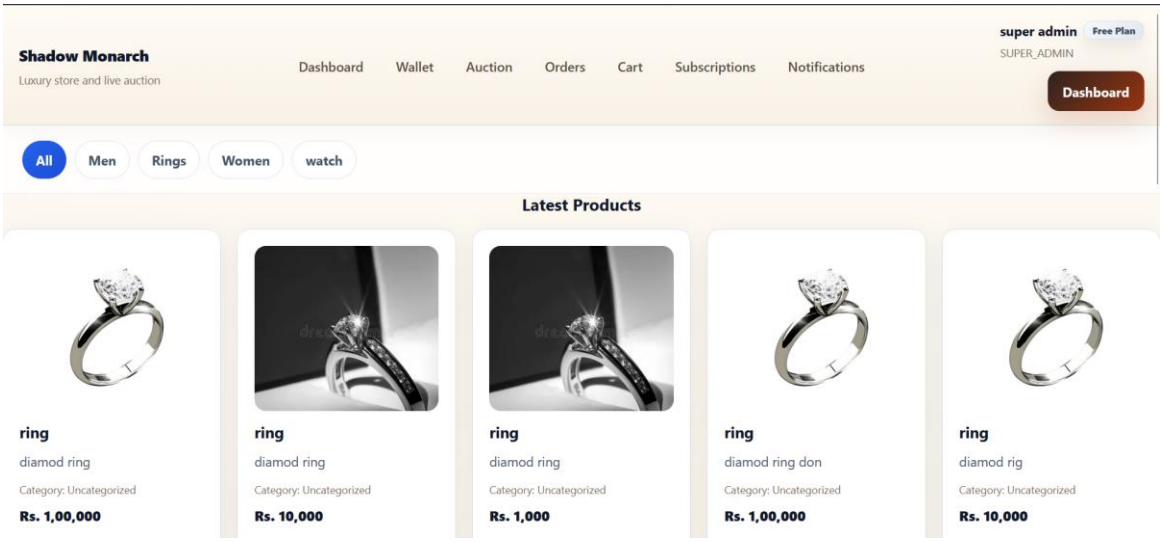


Figure 2.3.1 Home page showing featured products and main navigation.

The home interface combines direct shopping routes, customer shortcuts, and a product grid for the fashion marketplace.

Guest Product Browsing Screen

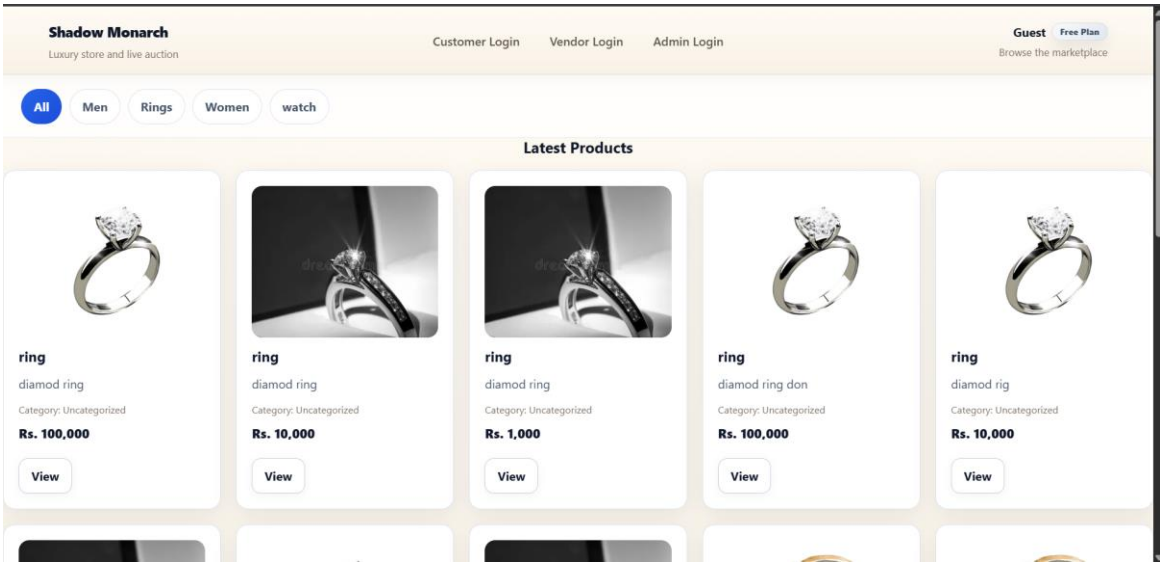


Figure 2.3.2 Guest browsing view with product cards and navigation links.

This screen supports product discovery before login and demonstrates that catalog browsing is available even for guest users.

Live Auction Dashboard Snapshot

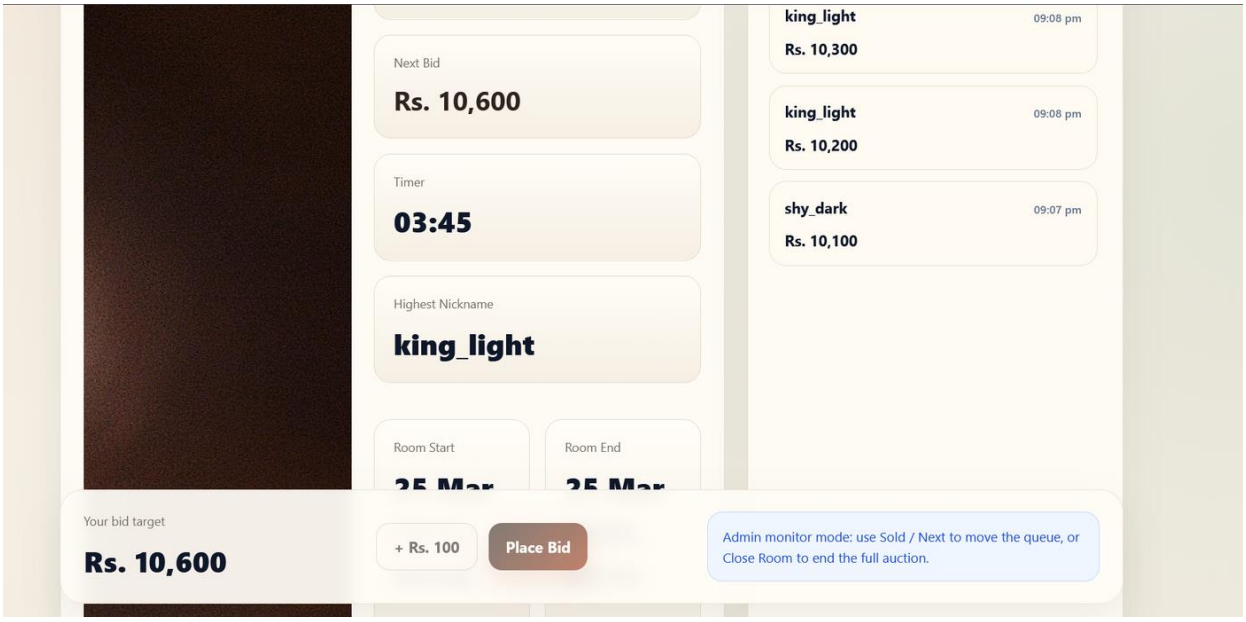


Figure 2.3.3 Customer live auction interface with timer, bid target, and recent bids.

The live room presents the current product, timer, next valid bid amount, and recent bid history in real time.

3. DEVELOPMENT ENVIRONMENT

3.1 HARDWARE REQUIREMENTS

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or above
RAM	8 GB minimum, 16 GB recommended
Storage	256 GB SSD or above
Display	1366 x 768 resolution or above
Network	Reliable broadband internet connection

3.2 SOFTWARE REQUIREMENTS

Software	Version / Purpose
Operating System	Windows 10/11 or equivalent development platform
Editor	Visual Studio Code
Database	MongoDB
Runtime Environment	Node.js
API Testing Tool	Postman
Browser	Google Chrome / Microsoft Edge
Version Control	Git

3.3 SOFTWARE SPECIFICATION

MongoDB is used as the document-oriented database for storing flexible but structured entities such as products, auctions, winners, carts, orders, and payments. Express.js provides the API layer that exposes route handlers for the application modules. React.js is used for the frontend interface, enabling reusable components, route-based page organization, and responsive user experiences. Node.js executes the server-side runtime and coordinates REST APIs, middleware, and real-time event communication.

The actual project structure reflects this stack. The frontend contains route-based pages for products, customers, vendors, wallet, orders, and auctions, while the backend is divided into modules such as product, category, cart, order, payment, auction, bidding,

winner, hosting, notification, and dashboard. This modular structure supports maintainability and future enhancement.

Project analysis shows that the running implementation is centered on MongoDB and Mongoose. The bundled SQL dump serves as an additional schema reference for academic documentation, comparative database discussion, and report preparation.

Layer	Technology	Purpose
Client	React 19 + Vite	SPA frontend for customer, admin, vendor, and auction pages
Communication	Axios + Socket.IO Client	REST and real-time bid updates
Server	Node.js + Express	API execution and business logic
Realtime	Socket.IO	Auction room synchronization
Database	MongoDB + Mongoose	Persistent storage of entities and references
Authentication	JWT + role middleware	Secure access for admin, vendor, and customer
Uploads	Multer	Image upload handling for product records

3.4 MODULE OVERVIEW

The backend is organized by feature modules, making the codebase easier to scale and maintain. Each business feature is implemented through a set of route, controller, service, and model files. This approach reduces coupling and supports academic analysis of the application as an enterprise-style software product.

The frontend also follows feature-based grouping, with separate pages for auction, product, order, customer, admin, vendor, wallet, category, payment, and notification workflows. Routing is centralized in the application entry file and protected routes are used for sensitive operations.

Module	Major Responsibility	Primary Users
Admin	Admin login, user management, approvals	Admin
Customer	Registration, login, profile and bank account	Customer
Vendor	Vendor onboarding and profile management	Vendor
Product	Catalog, product analytics, stock, search	Admin, Vendor, Customer
Cart	Item selection and quantity maintenance	Customer
Order	Order placement, tracking, cancellation, analytics	Customer, Admin, Vendor
Payment	Payment recording and order linking	Customer, Admin
Wallet	Cash/chips balance and history	Customer
Auction	Room creation, entry, state, finalization	Admin, Customer
Bidding	Bid history and leaderboard	Customer, Admin
Winner	Winner declaration and winner view	Admin, Customer
Notification	System alerts and unread count	All roles

4. SYSTEM DESIGN

4.1 DATA MODEL

4.1.1 ER DIAGRAM

The Entity Relationship model of the system consists of core entities such as Admin, Vendor, Customer, Category, Product, Cart, Order, Payment, Auction, Bid, Winner, Wallet, Review, and Notification. A vendor can create multiple products, a category can classify multiple products, a customer can maintain one cart with many items, and each order may contain multiple product items. Auction rooms are associated with one or more selected products, while bids and winner records are associated with customers and auction sessions. This model ensures that both direct sale and auction sale flows coexist without conflicting with each other.

• Vendor to Product: one-to-many.

• Category to Product: one-to-many.

• Customer to Cart: one-to-one or one-to-many over time.

• Customer to Order: one-to-many.

• Order to Payment: one-to-one or one-to-many based on payment logs.

• Auction to Winner: one-to-many for queued products.

• Customer to Bid/Winner: one-to-many.

4.1.2 LOGICAL DATA DESIGN

The logical design separates responsibilities into presentation, application, and persistence layers. The React client gathers user input and renders shopping and auction views. The Express application validates requests, applies authentication and authorization, invokes service logic, and coordinates database access. MongoDB stores business entities as collections with reference identifiers. Socket communication extends the logical design to support time-sensitive auction events such as room join, bid placement, state refresh, and fold actions.

4.1.2.1 CORE COLLECTIONS

The core collections participating in the day-to-day operation of the system are Product, Auction, Bid, Order, Payment, Wallet, Customer, Vendor, Winner, and

Notification. Product and Auction are especially important because the project is built on a dual-selling model where the same platform supports both direct sales and live bidding.

4.1.3 PHYSICAL DATA DESIGN

The physical data design converts the logical structure of the system into concrete MongoDB collections, document references, status fields, and timestamp-driven records that support both standard shopping and live auction processing. Collections such as Product, Category, Customer, Vendor, Cart, Order, Payment, Wallet, Auction, Bid, Winner, Review, and Notification are maintained separately so that the application can store business data in an organized and scalable manner. This design allows the platform to preserve product catalog details, bid history, order life cycle, wallet movement, and notification flow without mixing unrelated operations in a single structure.

In the physical layer, reference identifiers are used to connect customers with orders, orders with payments, vendors with products, and auction rooms with bids and winners. Additional operational fields such as room status, payment status, delivery status, created date, updated date, and auction start and end time support tracking, monitoring, and reporting. Index-oriented access on product identifiers, room codes, user references, and time-based fields improves retrieval efficiency and helps the system manage real-time bidding, settlement, and transaction verification in a dependable way.

4.1.4 DATA DICTIONARY

Table / Collection	Important Fields	Purpose
Product	vendor_id, category_id, product_name, price, stock, status, auction_room_id	Stores product master details
Auction	room_code, products, current_price, participants, start_time, end_time, status	Stores live auction room state
Winner	auction_id, product_id, user_id, winner_name, winning_bid, result_type	Stores declared winners

Order	customer_id, items, total_amount, status, delivery_deadline	Stores shopping orders
Payment	order_id, amount, payment_type, payment_status	Stores payment records
Cart	user_id, items, total_amount	Stores shopping cart data
Customer	name, email, password, wallet_balance, orders_count	Stores customer account data

4.1.5 TABLE RELATIONSHIP

Products are mapped to vendors and categories. Auctions hold product queue information and participant data. Winners refer to both auction and product identifiers for traceable settlement. Orders are linked with customers, while payments refer to orders. These relationships help maintain referential consistency at the application level, even within a NoSQL database environment.

Parent entity	Child entity	Relationship type	Description
Vendor	Product	One to many (1:N)	A vendor can own multiple fashion products
Category	Product	One to many (1:N)	A category can classify many products
Customer	Order	One to many (1:N)	A customer can place multiple orders
Order	Payment	One to many (1:N)	An order may have related payment records

Auction	Bid	One to many (1:N)	An auction room contains many bids
Auction	Winner	One to many (1:N)	A room can produce winners for multiple queued products
Customer	Wallet	One to one (1:1)	Each customer owns a single wallet record

4.2 PROCESS MODEL

4.2.1 CONTEXT ANALYSIS

The context analysis of the system identifies the major external actors as Administrator, Vendor, Customer, Payment Process, and MongoDB data store. Customers interact with the platform for product browsing, cart handling, checkout, wallet activity, and live auction participation. Vendors manage fashion products and monitor business operations, while administrators supervise user management, approvals, order monitoring, auction room control, and winner declaration

.

4.2.2 USE CASE DIAGRAM

The principal actors are Administrator, Vendor, and Customer. The Administrator manages customers, vendors, products, orders, auction rooms, hosting, and winner declaration. The Vendor maintains product inventory and order visibility. The Customer registers, logs in, browses products, adds items to cart, makes payments, joins auctions, places bids, folds from bidding, and views winners. These use cases represent the functional interaction scope of the project.

Use Case ID	Use Case Name	Actor	Description
UC-01	Register and login	Customer / Vendor / Admin	User authenticates into the system

UC-02	Manage products	Admin / Vendor	Create, update, search, and remove products
UC-03	Manage categories	Admin	Create and maintain product categories
UC-04	Add to cart	Customer	Add product and update quantity
UC-05	Place order	Customer	Checkout selected cart items
UC-06	Make payment	Customer	Pay for an order using supported methods
UC-07	Add wallet money	Customer	Credit wallet balance for future use
UC-08	Create auction room	Admin	Prepare auction room and assign items
UC-09	Join auction	Customer	Enter active room and view room state
UC-10	Place bid	Customer	Bid on live auction product
UC-11	Finalize product	Admin	Mark current auction product sold or move next
UC-12	View winners	Customer / Admin	Display winner list and winning bid

4.2.3 DATA FLOW DIAGRAM (DFD)

In the context-level Data Flow Diagram, users interact with the Fashion E-Commerce with Auction System to submit registration details, browse products, place orders, deposit wallet amounts, join auction rooms, and submit bids. The system communicates with the MongoDB database to store and retrieve customer records, product

details, order history, payment logs, auction states, and winner information. In lower-level DFDs, data moves through modules such as authentication, product management, order processing, wallet management, and live auction control.

Entity Relationship Diagram

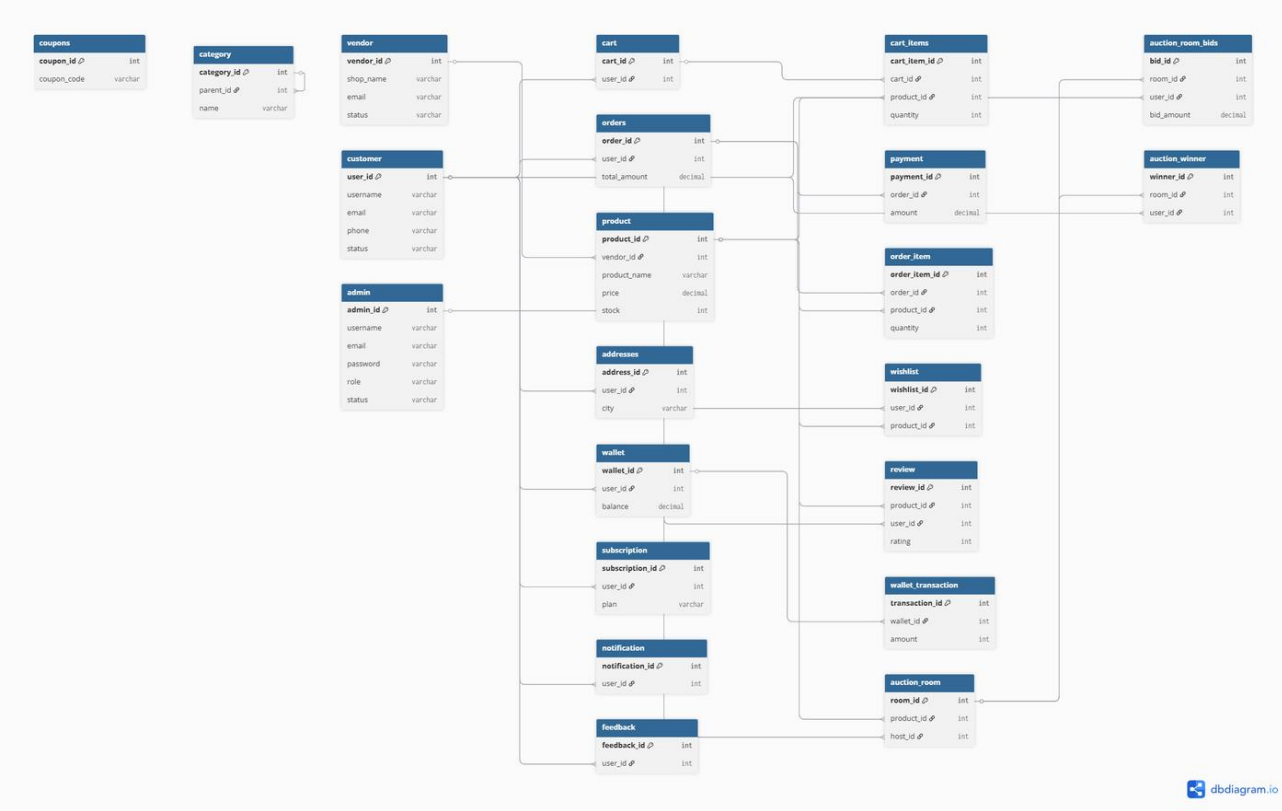


Figure 4.1.1 Entity Relationship Diagram prepared from the project data assets.

The ER figure consolidates the main commerce and auction entities, including users, products, wallet records, orders, payments, auctions, bids, winners, and notifications.

Use Case Diagram

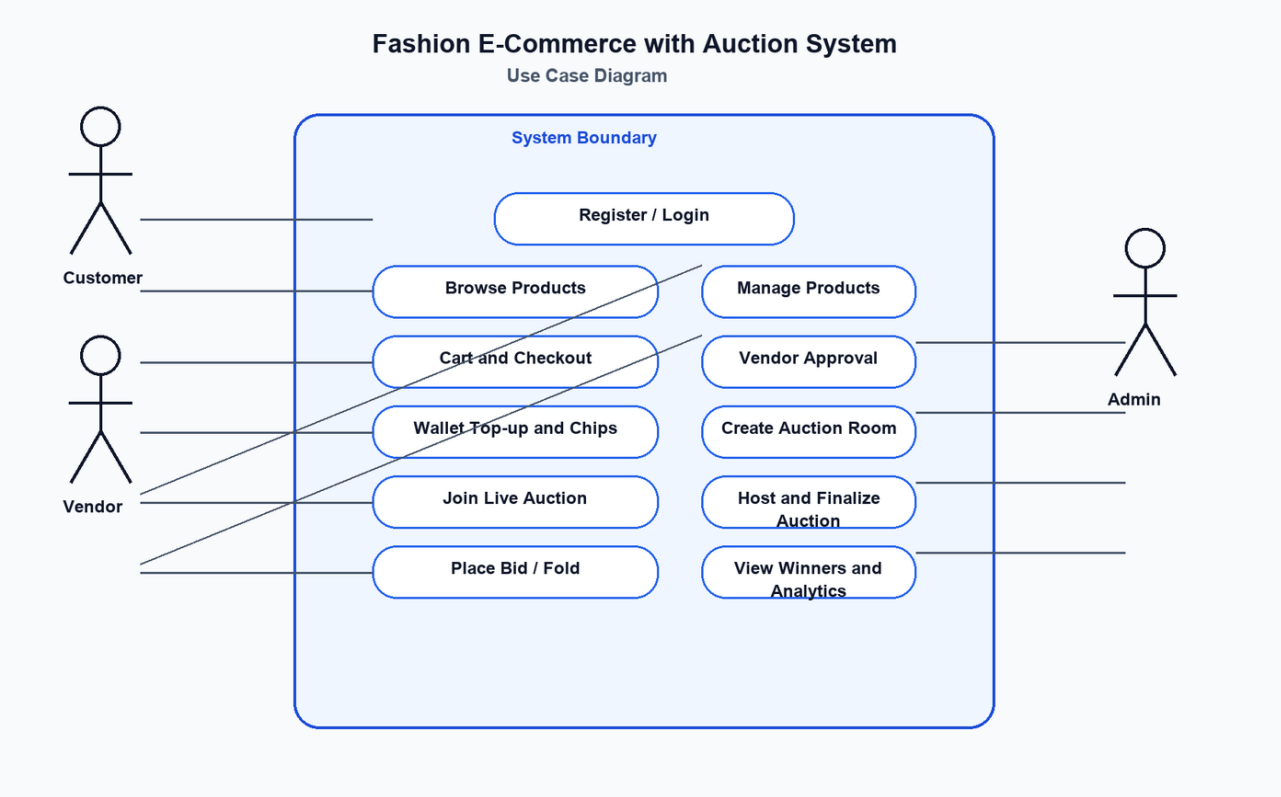


Figure 4.2.2 Use case diagram derived from the implemented admin, vendor, and customer flows.

The diagram highlights how the three main actors interact with catalog browsing, wallet operations, auction entry, room hosting, and analytics workflows.

DFD Level 0

Figure 4.2.3(a) DFD Level 0 for the Fashion E-Commerce with Auction System.

Level 0 describes the highest-level interaction among the user roles, the platform, and the data store.

DFD Level 1

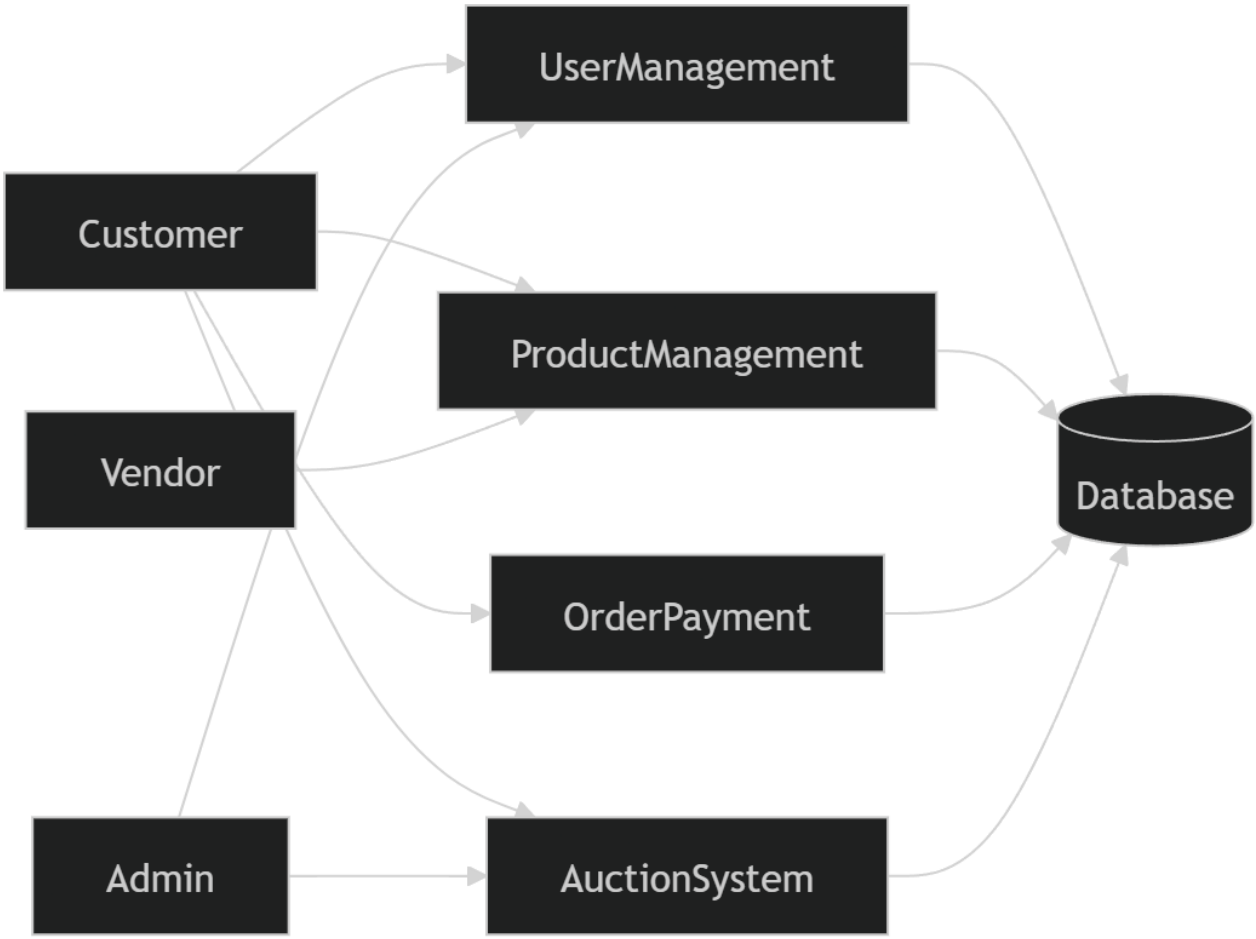


Figure 4.2.3(b) DFD Level 1 showing the first decomposition of the main process.

Level 1 expands the core process into detailed commerce and auction sub-processes.

DFD Level 2

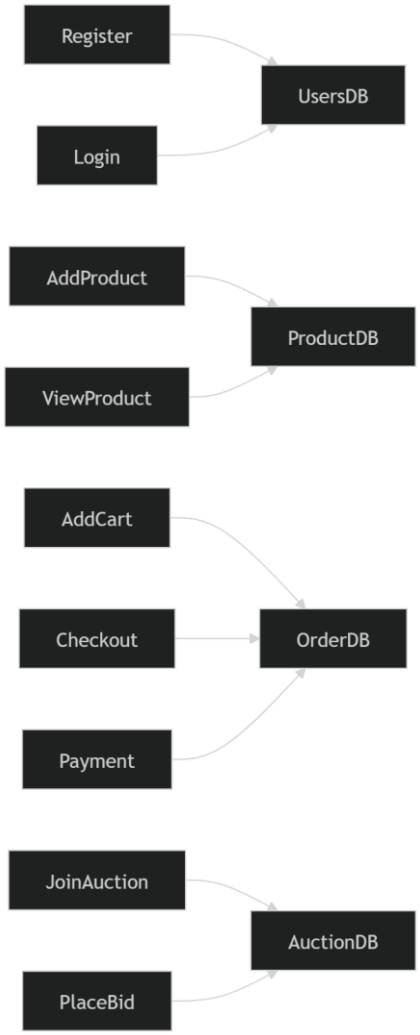


Figure 4.2.3(c) DFD Level 2 showing refined module interactions.

Level 2 highlights intermediate flows among account, product, order, wallet, and auction operations.

DFD Level 3

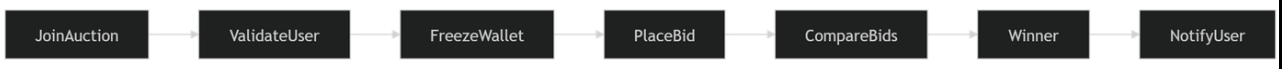


Figure 4.2.3(d) DFD Level 3 showing detailed operational flow.

Level 3 captures the lowest-level movement of data for real-time bidding and transaction processing.

4.2.4 DECISION TREE

Auction participation follows a clear decision structure. If the user is authenticated and is a customer, the system checks whether the room is live, whether the user has sufficient wallet balance, and whether the current bid amount is valid. If all conditions are

satisfied, the bid is accepted; otherwise, the system rejects the request or marks the participant as folded. This rule set can be represented as a decision tree for process flow and as a decision table for implementation clarity.

4.2.5 DECISION TABLE

Condition	Yes	No
Authenticated customer?	Proceed to room validation	Reject access
Auction room live?	Proceed to wallet check	Display room unavailable
Wallet balance sufficient?	Proceed to bid validation	Fold or block bidder
Bid amount valid?	Accept bid and broadcast	Reject bid

5. ARCHITECTURAL DETAILS

5.1 PROGRAM DESIGN LANGUAGE

The program design language section describes the high-level operational logic of the project in structured pseudo code. The objective is to show the sequence of actions performed by the system from startup to transaction completion, including direct shopping and auction-specific workflows.

AUCTION WORKFLOW PSEUDO CODE

```
START
  LOGIN USER
  IF role = CUSTOMER THEN
    LOAD products and wallet details
    IF user chooses direct purchase THEN
      add item to cart
      create order
      process payment
    ELSE IF user joins auction room THEN
      validate room status
      validate wallet balance
      WHILE room is live
        display current product and next bid
        IF bid submitted AND valid THEN
          store bid
          update current price
          broadcast room state
        ELSE IF customer folds THEN
          update participant status
        END IF
      END WHILE
      declare winner
    END IF
  ELSE IF role = ADMIN THEN
    create room
    assign products
    monitor live room
    finalize sold product or close room
  END IF
STOP
```

ORDER PROCESSING PSEUDO CODE

```
BEGIN OrderCheckout
  FETCH cart items by user
  IF cart is empty THEN
    SHOW 'No items to checkout'
  ELSE
    CALCULATE grand total
    CREATE order record
    IF payment succeeds THEN
      UPDATE payment status as SUCCESS
      UPDATE order status as PAID
```

```
      REDUCE product stock
    ELSE
      MARK payment as FAILED
    END IF
  END IF
END
```

WALLET TOP-UP PSEUDO CODE

```
BEGIN AddWalletMoney
  INPUT user_id, amount
  VALIDATE amount > 0
  FETCH wallet by user_id
  IF wallet does not exist THEN
    CREATE wallet
  END IF
  INCREMENT cash balance
  APPEND CREDIT transaction history
  RETURN updated wallet
END
```

5.2 N-TIER ARCHITECTURE

The project follows a three-tier architecture. The presentation tier is the React frontend, which contains route-driven pages and components for shopping, profile management, payment, wallet operations, and live bidding. The application tier is the Node.js and Express backend that hosts REST APIs, authentication middleware, validation, service logic, and Socket.IO communication. The data tier is MongoDB, which stores persistent business entities. This layered separation improves scalability, maintainability, and clarity of responsibility.

• Presentation Tier: React pages, forms, dashboards, and live room UI.

• Application Tier: Express routes, controllers, services, middleware, and sockets.

• Data Tier: MongoDB collections for users, products, orders, payments, auctions, bids, and winners.

Tier	Files / Components	Responsibilities
Presentation	App.jsx, pages/*, components/*	UI rendering, user interaction, navigation
Business Logic	controllers, services, middleware	Validation, workflow control, role checks
Persistence	models/*.js, MongoDB	Schema design and permanent storage
Realtime Layer	server.js + socket.io-client	Live bid synchronization and room state updates

System Architecture Diagram

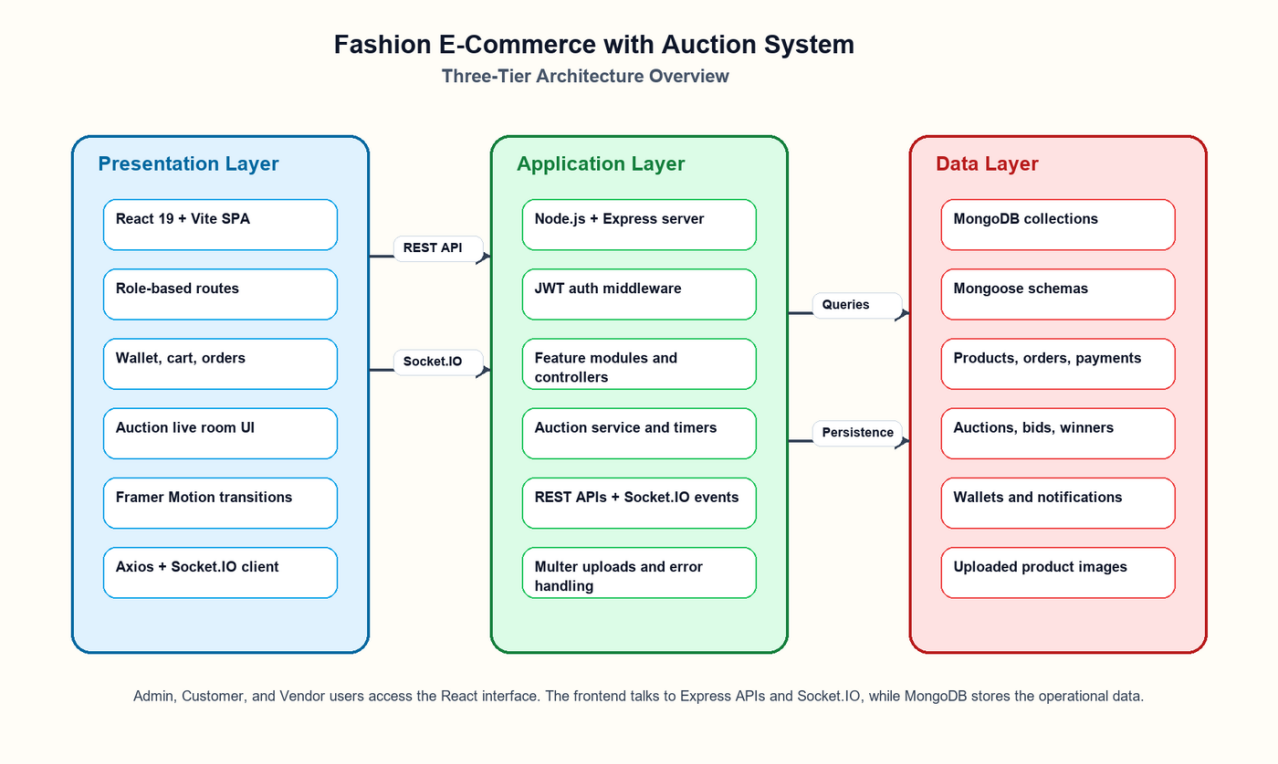


Figure 5.2.1 Three-tier architecture of the implemented project.

The frontend, Express application layer, and MongoDB persistence layer are connected through REST APIs, uploads, and Socket.IO live events.

6. SYSTEM IMPLEMENTATION

6.1 CLIENT-SIDE CODING

The frontend is developed using React and is organized into pages and reusable components. Route management is handled through React Router. The application provides separate interfaces for customer, vendor, and administrator operations. Product pages, cart management, checkout, payment status, wallet pages, order dashboards, and auction-specific pages are available through dedicated routes. The interface also uses lazy loading and transition wrappers to improve usability and navigation flow.

Auction participation on the frontend is implemented through a live room component that loads room state from the API, connects to the server using Socket.IO, shows the current price and timer, and allows the customer to place bids or fold from the room. The component also displays recent bids, sold products, and active participants in real time.

FRONTEND ROUTE ORGANIZATION

The central application router contains paths for customer registration and login, admin management, vendor management, product dashboards, order pages, wallet flows, notification pages, and a complete auction section. This demonstrates that the project is not a limited proof-of-concept, but rather a multi-role business application.

APP.JSX ROUTE SNIPPET

```
import { Suspense, lazy } from "react";
import { BrowserRouter, Route } from "react-router-dom";
import AppShell from "./components/layout/AppShell";
import ProtectedRoute from "./components/ProtectedRoute";
import RouteAnimator from "./components/ui/RouteAnimator";
import PageTransition from "./components/ui/PageTransition";
import LoadingSpinner from "./components/ui/LoadingSpinner";
const Home = lazy(() => import("./pages/Home"));
import AddCustomer from "./pages/customer/AddCustomer";
import AllCustomers from "./pages/customer/AllCustomers";
import BlockedCustomers from "./pages/customer/BlockedCustomers";
import CustomerAnalytics from "./pages/customer/CustomerAnalytics";
import Dashboard from "./pages/customer/Dashboard";
import Profile from "./pages/customer/Profile";
import SearchCustomer from "./pages/customer/SearchCustomer";
import UpdateCustomer from "./pages/customer/UpdateCustomer";
import CustomerDashboard from "./pages/customer/customerDashboard";
import CustomerLogin from "./pages/customer/customerlogin";
import CustomerRegister from "./pages/customer/customerregister";
import Wallet from "./pages/wallet/wallet";
const AddMoney = lazy(() => import("./pages/wallet/AddMoney"));
```

```
const WalletBalance = lazy(() =>
import("./pages/wallet/WalletBalance"));
const WalletHistory = lazy(() =>
import("./pages/wallet/WalletHistory"));
const BankAccount = lazy(() => import("./pages/wallet/BankAccount"));
import AdminDashboard from "./pages/admin/AdminDashboard";
import AdminLogin from "./pages/admin/AdminLogin";
import AdminOrders from "./pages/admin/AdminOrders";
import AdminProfile from "./pages/admin/AdminProfile";
import AdminRegister from "./pages/admin/AdminRegister";
import AddVendor from "./pages/vendor/AddVendor";
import AllVendors from "./pages/vendor/AllVendors";
import BlockedVendors from "./pages/vendor/BlockedVendors";
import SearchVendor from "./pages/vendor/SearchVendor";
import UpdateVendor from "./pages/vendor/UpdateVendor";
import VendorAnalytics from "./pages/vendor/VendorAnalytics";
import VendorApproval from "./pages/vendor/VendorApproval";
import VendorDashboard from "./pages/vendor/Dashboard";
import VendorPanel from "./pages/vendor/vendorDashboard";
import VendorLogin from "./pages/vendor/vendorlogin";
import VendorRegister from "./pages/vendor/vendorregister";
import Cart from "./pages/cart/cart";
import Category from "./pages/category/Category";
import AddCategory from "./pages/category/addcategory";
const Orders = lazy(() => import("./pages/orders/orders"));
import Subscription from "./pages/Subscription/Subscription";
import Payment from "./pages/payment/payment";
import PaymentFailed from "./pages/payment/PaymentFailed";
import PaymentProcessing from "./pages/payment/PaymentProcessing";
import PaymentSuccess from "./pages/payment/PaymentSuccess";
import ShopCheckout from "./pages/checkout/Checkout";
import AllOrders from "./pages/orders/AllOrders";
import CancelledOrders from "./pages/orders/CancelledOrders";
import CancelOrder from "./pages/orders/CancelOrder";
import OrderAnalytics from "./pages/orders/OrderAnalytics";
import OrderCheckout from "./pages/orders/Checkout";
import OrderDashboard from "./pages/orders/OrderDashboard";
import OrderDetails from "./pages/orders/OrderDetails";
import OrderSuccess from "./pages/orders/OrderSuccess";
import UpdateOrderStatus from "./pages/orders/UpdateOrderStatus";
import VendorOrderDetails from "./pages/orders/VendorOrderDetails";
import VendorOrders from "./pages/orders/VendorOrders";
import VendorSalesAnalytics from "./pages/orders/VendorSalesAnalytics";
import Notifications from "./pages/notification/notification";
import AddProduct from "./pages/Product/AddProduct";
import AllProduct from "./pages/Product/AllProduct";
import LowStockProducts from "./pages/Product/LowStockProducts";
import ProductAnalytics from "./pages/Product/ProductAnalytics";
import ProductDashboard from "./pages/Product/ProductDashboard";
const ProductDetails = lazy(() =>
import("./pages/Product/ProductDetails"));
import TopRatedProducts from "./pages/Product/TopRatedProducts";
import TopSellingProducts from "./pages/Product/TopSellingProducts";
import UpdateProduct from "./pages/Product/UpdateProduct";
import AuctionDashboard from "./pages/auction/AuctionDashboard";
const AuctionAddProduct = lazy(() =>
import("./pages/auction/AddProduct"));
import AuctionLiveDashboard from "./pages/auction/AuctionLiveDashboard";
const AuctionProductLimit = lazy(() =>
import("./pages/auction/AuctionProductLimit"));
const AuctionProductSelection = lazy(() =>
```

```
import("./pages/auction/AuctionProductSelection"));
const CreateAuctionRoom = lazy(() =>
import("./pages/auction/CreateAuctionRoom"));
import Hosting from "./pages/auction/Hosting";
import HostingManager from "./pages/auction/HostingManager";
import LiveBidding from "./pages/auction/livebidding";
import RoomsList from "./pages/auction/RoomsList";
import WinnerList from "./pages/auction/winnerlist";
import AuctionUserPage from "./pages/auction/AuctionUserPage";
import JoinRoom from "./pages/auction/JoinRoom";
import WinnerPage from "./pages/auction/WinnerPage";

const renderLazy = (element) => (
  <Suspense fallback={<div className="shop-shell"><div className="shop-
container"><LoadingSpinner centered label="Loading page..."
/></div></div>}>
    {element}
  </Suspense>
);

export default function App() {
  return (
    <BrowserRouter>
      <AppShell>
        <RouteAnimator>
          <Route path="/" element={<PageTransition>{renderLazy(<Home
/>)}</PageTransition>} />
          <Route path="/customer/login"
element={<PageTransition><CustomerLogin /></PageTransition>} />
          <Route path="/customer/register"
element={<PageTransition><CustomerRegister /></PageTransition>} />
          <Route path="/customer/profile"
element={<PageTransition><Profile /></PageTransition>} />
          <Route path="/customer/dashboard"
element={<PageTransition><CustomerDashboard /></PageTransition>} />
          <Route path="/admin/customers"
element={<PageTransition><Dashboard /></PageTransition>} />
          <Route path="/admin/customers/all"
element={<PageTransition><AllCustomers /></PageTransition>} />
          <Route path="/admin/customers/add"
element={<PageTransition><AddCustomer /></PageTransition>} />
          <Route path="/admin/customers/update/:id"
element={<PageTransition><UpdateCustomer /></PageTransition>} />
          <Route
path="/subscriptions"element={<PageTransition><Subscription
/></PageTransition>}/>
          <Route path="/admin/customers/blocked"
element={<PageTransition><BlockedCustomers /></PageTransition>} />
          <Route path="/admin/customers/analytics"
element={<PageTransition><CustomerAnalytics /></PageTransition>} />
          <Route path="/admin/customers/search"
element={<PageTransition><SearchCustomer /></PageTransition>} />
          <Route path="/wallet" element={<PageTransition><Wallet
/></PageTransition>} />
          <Route path="/wallet/add-money"
element={<PageTransition>{renderLazy(<AddMoney />)}</PageTransition>} />
          <Route path="/wallet/history"
element={<PageTransition>{renderLazy(<WalletHistory
/>)}</PageTransition>} />
          <Route path="/wallet/walletbalance"
element={<PageTransition>{renderLazy(<WalletBalance
```

```
/>)}</PageTransition>} />
          <Route path="/wallet/bank-account"
element={<PageTransition><ProtectedRoute>{renderLazy(<BankAccount
/>)}</ProtectedRoute></PageTransition>} />
          <Route path="/admin/login"
element={<PageTransition><AdminLogin /></PageTransition>} />
          <Route path="/admin/register"
element={<PageTransition><AdminRegister /></PageTransition>} />
          <Route path="/admin/dashboard"
element={<PageTransition><AdminDashboard /></PageTransition>} />
          <Route path="/admin/profile"
element={<PageTransition><AdminProfile /></PageTransition>} />
```

Frontend Navigation Flow

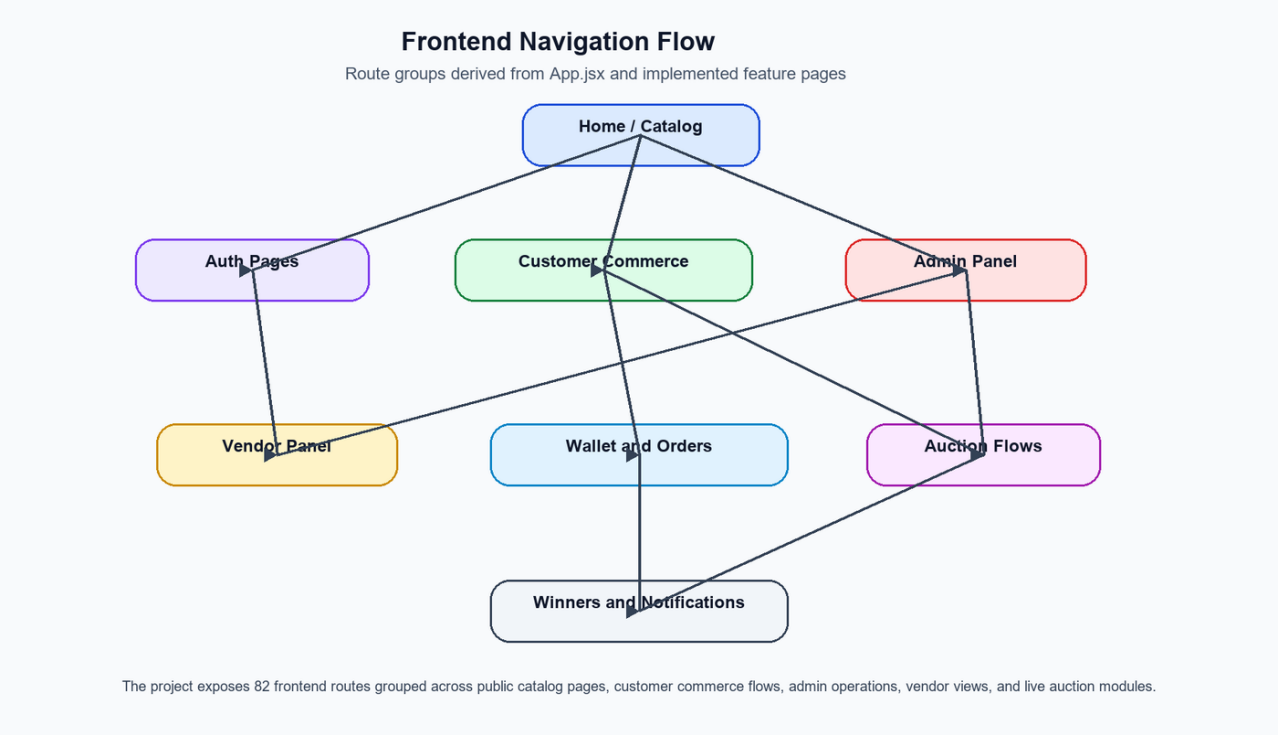


Figure 6.1.1 High-level navigation flow based on the routes defined in App.jsx.

The route map reflects 82 implemented frontend routes across public pages, customer commerce, admin operations, vendor views, and auction modules.

LIVE AUCTION FRONTEND COMPONENT

The LiveAuctionRoom component acts as the real-time interaction center of the project. It receives room state, tracks recent bids, maintains countdown timers, validates bid placement availability, and updates the interface immediately when socket events are received from the server.

LIVEAUCTIONROOM.JSX SNIPPET

```
import { useCallback, useEffect, useMemo, useRef, useState } from
"react";
import { io } from "socket.io-client";
import { useNavigate } from "react-router-dom";
import { useToast } from "../ui/ToastProvider";
import LoadingSpinner from "../ui/LoadingSpinner";
import api, { API_BASE_URL } from "../../utils/axios";
import { showResultAlert } from "../../utils/alerts";

const SOCKET_BASE_URL = API_BASE_URL.replace(/\/api$/, "");

const getStoredToken = () =>
  localStorage.getItem("accessToken") ||
  localStorage.getItem("token") ||
  sessionStorage.getItem("accessToken") ||
  sessionStorage.getItem("token") ||
  "";

const formatMoney = (value) => `Rs. ${Number(value ||
0).toLocaleString()}`;

const formatCountdown = (seconds) => {
  const safeSeconds = Math.max(0, Number(seconds || 0));
  const minutes = Math.floor(safeSeconds / 60);
  const remainingSeconds = safeSeconds % 60;
  return `${String(minutes).padStart(2,
"0")}:${String(remainingSeconds).padStart(2, "0")}`;
};

const formatDateTime = (value) => {
  if (!value) {
    return "Not scheduled";
  }

  const parsed = new Date(value);

  if (Number.isNaN(parsed.getTime())) {
    return "Not scheduled";
  }

  return parsed.toLocaleString("en-IN", {
    year: "numeric",
    month: "short",
    day: "2-digit",
    hour: "2-digit",
    minute: "2-digit"
  });
};

const formatBidTime = (value) => {
  if (!value) {
    return "Just now";
  }

  const parsed = new Date(value);

  if (Number.isNaN(parsed.getTime())) {
    return "Just now";
```

```
  }

  return parsed.toLocaleTimeString("en-IN", {
    hour: "2-digit",
    minute: "2-digit"
  });
};

export default function LiveAuctionRoom({ auctionId, mode = "customer"
}) {
  const navigate = useNavigate();
  const toast = useToast();
  const socketRef = useRef(null);
  const prevRoomRef = useRef(null);
  const removalAlertShown = useRef(false);
  const priceTimerRef = useRef(null);

  const user = useMemo(() => {
    try {
      return JSON.parse(localStorage.getItem("user") || "null");
    } catch {
      return null;
    }
  }, []);

  const [room, setRoom] = useState(null);
  const [recentBids, setRecentBids] = useState([]);
  const [loading, setLoading] = useState(true);
  const [connecting, setConnecting] = useState(true);
  const [pendingBid, setPendingBid] = useState(0);
  const [placingBid, setPlacingBid] = useState(false);
  const [pricePulse, setPricePulse] = useState(false);
  const [connectionLabel, setConnectionLabel] = useState("Connecting");
  const [countdown, setCountdown] = useState(0);
  const [adminActionLoading, setAdminActionLoading] = useState(false);
  const [foldingBid, setFoldingBid] = useState(false);

  const isAdminView = mode === "admin";
  const canBid = !isAdminView && room?.viewer_can_bid;
  const removalMessage = room?.removal_message || "";
  const currentUserId = String(user?.user_id || user?._id || "");
  const currentProductStatus = room?.product?.status || "";
  const isProductSold = currentProductStatus === "SOLD";

  const applyRoomUpdate = useCallback((nextRoom) => {
    const previousRoom = prevRoomRef.current;

    if (previousRoom?.current_price !== nextRoom.current_price) {
      setPricePulse(true);
      window.clearTimeout(priceTimerRef.current);
      priceTimerRef.current = window.setTimeout(() =>
setPricePulse(false), 650);
    }

    if (
      nextRoom.viewer?.status === "FOLDED" &&
      !removalAlertShown.current &&
      nextRoom.removal_message !== "You folded from the current product"
    ) {
      removalAlertShown.current = true;
      toast.error("You are removed due to insufficient balance",
```

```
"Auction access removed");
      showResultAlert({
        title: "Auction Access Removed",
        text: "You are removed due to insufficient balance",
        icon: "warning",
        confirmButtonText: "Okay"
      });
    }

    setRoom(nextRoom);
    setCountdown(Number(nextRoom.countdown_seconds || 0));
    setPendingBid((current) => {
      if (!current || current < nextRoom.next_bid_amount) {
        return nextRoom.next_bid_amount;
      }
      return current;
    });
    prevRoomRef.current = nextRoom;
  }, [toast]);

  useEffect(() => {
    let active = true;

    const loadRoom = async () => {
      try {
        const [roomRes, bidsRes] = await Promise.all([
          api.get(`/auction/room/${auctionId}/state`),
          api.get(`/auction/board/${auctionId}`)
        ]);
        if (!active) return;
        applyRoomUpdate(roomRes.data.data);
        setRecentBids(
          (bidsRes.data.bids || []).map((bid) => ({
            id: bid._id,
            userId: String(bid.bidder?.user_id || ""),
            nickname: bid.bidder?.nickname || "Bidder",
            amount: Number(bid.bid_amount || 0),
            time: bid.time
          }))
        );
      } catch (error) {
        toast.error(error.response?.data?.message || "Unable to load
this auction room.");
      } finally {
        if (active) {
          setLoading(false);
        }
      }
    };

    loadRoom();

    return () => {
      active = false;
    };
  }, [applyRoomUpdate, auctionId, toast]);

  useEffect(() => {
    const interval = window.setInterval(async () => {
      try {
        const res = await api.get(`/auction/room/${auctionId}/state`);
```

```
        applyRoomUpdate(res.data.data);
      } catch (error) {
        console.error("Auction polling error:", error);
      }
    }, 2000);

    return () => window.clearInterval(interval);
  }, [applyRoomUpdate, auctionId]);

  useEffect(() => {
    const token = getStoredToken();
    const socket = io(SOCKET_BASE_URL, {
      transports: ["websocket"],
      auth: token ? { token } : undefined
    });

    socketRef.current = socket;

    socket.on("connect", () => {
      setConnectionLabel("Live");
      setConnecting(false);
      socket.emit("auction:join", { auctionId }, (response) => {
        if (response?.success && response.data) {
          applyRoomUpdate(response.data);
          return;
        }

        if (response?.message) {
          toast.error(response.message);
        }
      });
    });

    socket.on("disconnect", () => {
      setConnectionLabel("Reconnecting");
      setConnecting(true);
    });

    socket.on("auction:state", (nextRoom) => {
      applyRoomUpdate(nextRoom);

      if (nextRoom.status === "ENDED") {
        toast.success("Auction room completed.");
      }
    });

    socket.on("auction:bid-placed", (payload) => {
```

Live Auction UI

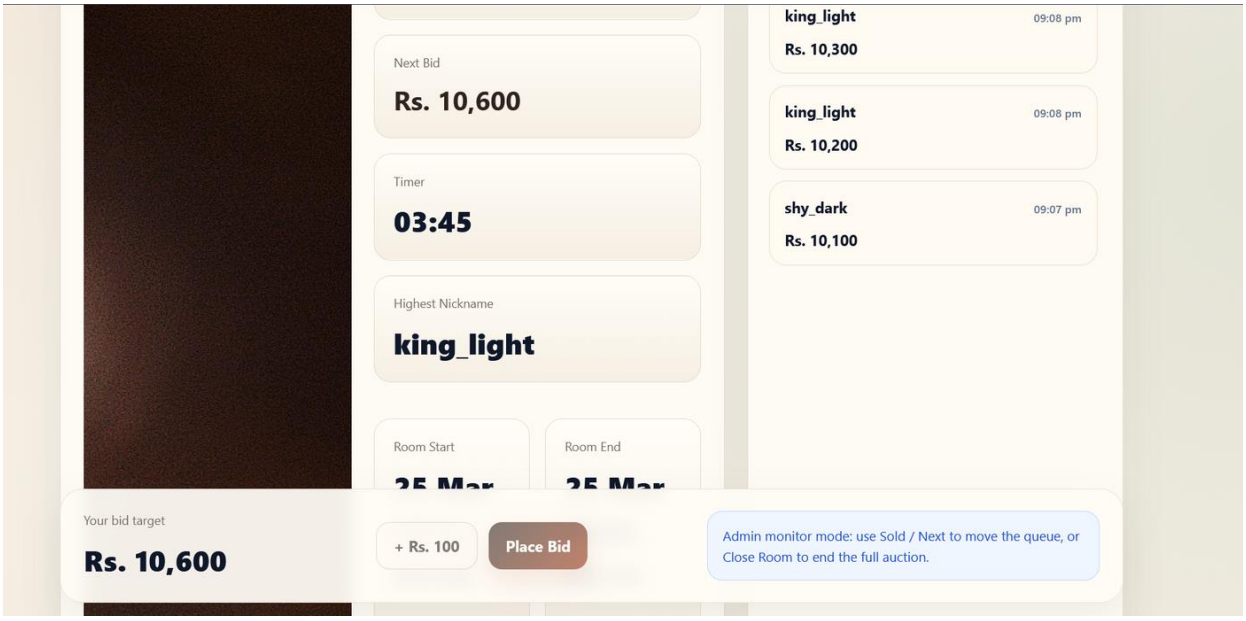


Figure 6.1.2 Live auction screen with bid amount, timer, bidder history, and room details.

The UI confirms that the customer-facing bidding experience is fully integrated into the same project rather than handled by a separate prototype.

6.2 SERVER-SIDE CODING

The backend follows a modular implementation pattern. Independent modules exist for admin, customer, vendor, product, category, cart, order, payment, wallet, auction, bidding, winner, review, notification, and subscription management. Each module contains route definitions, models, and controllers. The server initializes database connectivity, exposes REST API endpoints, hosts uploaded resources, and establishes Socket.IO events for live auction synchronization. MongoDB with Mongoose is used for schema definition, validation, and data persistence.

A notable implementation detail is the use of real-time events such as auction join, bid, fold, and chat. These events help all participants view synchronized auction states without manual refresh and make the application suitable for live bidding use cases.

SERVER BOOTSTRAP AND ROUTE REGISTRATION

SERVER.JS SNIPPET

```
require("dotenv").config();

const express = require("express");
const cors = require("cors");
```

```
const path = require("path");
const http = require("http");
const { Server } = require("socket.io");
const jwt = require("jsonwebtoken");

const connectDB = require("./config/db");
const auctionService = require("./modules/auction/auction.service");
const Auction = require("./modules/auction/auction.model");
const { setIo } = require("./socket/socketStore");
const { normalizeDecodedUser } = require("./middleware/auth");

const app = express();
const server = http.createServer(app);

connectDB();

app.use(cors({
  origin: process.env.CLIENT_URL || true,
  credentials: true
}));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.use("/uploads", express.static(path.join(__dirname, "uploads")));

app.use("/api/admin", require("./modules/admin/admin.routes"));
app.use("/api/customer", require("./modules/customer/customer.routes"));
app.use("/api/vendor", require("./modules/vendor/vendor.routes"));
app.use("/api/product", require("./modules/product/product.routes"));
app.use("/api/category", require("./modules/category/category.routes"));
app.use("/api/cart", require("./modules/cart/cart.routes"));
app.use("/api/order", require("./modules/order/order.routes"));
app.use("/api/payment", require("./modules/payment/payment.routes"));
app.use("/api/wallet", require("./modules/wallet/wallet.routes"));
app.use("/api/auction", require("./modules/auction/auction.routes"));
app.use("/api/bidding", require("./modules/bidding/bidding.routes"));
app.use("/api/winner", require("./modules/winner/winner.routes"));
app.use("/api/hosting", require("./modules/hosting/hosting.routes"));
app.use("/api/subscription",
require("./modules/Subscription/subscription.routes"));
app.use("/api/notification",
require("./modules/notification/notification.routes"));
app.use("/api/notifications",
require("./modules/notification/notification.routes"));
app.use("/api/reviews", require("./modules/reviews/review.routes"));
app.use("/api/wishlist", require("./modules/wishlist/wishlist.routes"));
app.use("/api/dashboard",
require("./modules/dashboard/dashboard.routes"));
app.use("/api/upload", require("./modules/upload/upload.routes"));

app.get("/", (req, res) => {
  res.json({
    success: true,
    message: "Fashion Auction API Running"
  });
});

app.use(require("./middleware/errorHandler"));

const PORT = process.env.PORT || 5001;
const io = new Server(server, {
```

```
  cors: {
    origin: process.env.CLIENT_URL || true,
    credentials: true
  }
});

setIo(io);

Auction.find({ status: "LIVE" })
  .select("_id")
  .then((auctions) => Promise.all(auctions.map((auction) =>
auctionService.ensureAuctionTimer(auction._id))))
  .catch(() => {});

io.use((socket, next) => {
  const token = socket.handshake.auth?.token ||
socket.handshake.headers?.authorization?.replace(/^Bearer\s+/i, "");

  if (!token) {
    socket.user = null;
    return next();
  }

  const secrets = [
    process.env.JWT_SECRET_ADMIN,
    process.env.JWT_SECRET_CUSTOMER,
    process.env.JWT_SECRET_VENDOR
  ];

  for (const secret of secrets) {
    try {
      socket.user = normalizeDecodedUser(jwt.verify(token, secret));
      return next();
    } catch (error) {}
  }

  return next(new Error("Invalid token"));
});

io.on("connection", (socket) => {
  socket.on("auction:join", async ({ auctionId }, callback = () => {})
=> {
    try {
      socket.join(`${auctionService.roomPrefix}${auctionId}`);

      let result;
      if (socket.user?.role === "CUSTOMER") {
        result = await
auctionService.enterAuctionService(socket.user.id, auctionId);
      } else {
        result = await auctionService.getRoomStateService(auctionId,
socket.user?.id || null);
      }

      if (!result.success) {
        callback(result);
        return;
      }

      callback({ success: true, data: result.data });
      await auctionService.emitAuctionState(auctionId, socket.user?.id
```

```
|| null);
    } catch (error) {
      callback({ success: false, message: error.message });
    }
  });

  socket.on("auction:bid", async ({ auctionId, bidAmount }, callback =
() => {}) => {
    try {
      if (!socket.user?.id || socket.user?.role !== "CUSTOMER") {
        callback({ success: false, message: "Only customers can bid."
});
        return;
      }

      const result = await
auctionService.placeBidService(socket.user.id, auctionId, bidAmount);
      callback(result);

      if (result.success) {

io.to(`${auctionService.roomPrefix}${auctionId}`).emit("auction:bid-
placed", {
          bidder_name: result.data.room.highest_bidder_name,
          current_price: result.data.room.current_price
        });
        await auctionService.emitAuctionState(auctionId,
socket.user.id);
      }
    } catch (error) {
      callback({ success: false, message: error.message });
    }
  });

  socket.on("auction:fold", async ({ auctionId }, callback = () => {})
=> {
    try {
      if (!socket.user?.id || socket.user?.role !== "CUSTOMER") {
        callback({ success: false, message: "Only customers can fold."
});
        return;
      }

      const result = await
auctionService.foldBidderService(socket.user.id, auctionId);
      callback(result);

      if (result.success) {
        await auctionService.emitAuctionState(auctionId,
socket.user.id);
      }
    } catch (error) {
      callback({ success: false, message: error.message });
    }
  });

  socket.on("auction:chat", async ({ auctionId, message }, callback = ()
=> {}) => {
    try {
      if (!socket.user?.id || socket.user?.role !== "CUSTOMER") {
        callback({ success: false, message: "Only customers can chat."
```

```
});
        return;
      }

      const result = await
auctionService.addChatMessageService(socket.user.id, auctionId,
message);
      callback(result);

      if (result.success) {

io.to(`${auctionService.roomPrefix}${auctionId}`).emit("auction:chat-
message", result.data);
        await auctionService.emitAuctionState(auctionId,
socket.user.id);
      }
```

AUCTION MODEL

AUCTION.MODEL.JS SNIPPET

```
const mongoose = require("mongoose");

const ParticipantSchema = new mongoose.Schema({
  user_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Customer",
    required: true
  },
  username: {
    type: String,
    required: true
  },
  wallet_balance: {
    type: Number,
    default: 0
  },
  status: {
    type: String,
    enum: ["ACTIVE", "FOLDED"],
    default: "ACTIVE"
  },
  folded_reason: {
    type: String,
    default: ""
  },
  last_bid_amount: {
    type: Number,
    default: 0
  },
  joined_at: {
    type: Date,
    default: Date.now
  },
  folded_at: Date
}, { _id: false });

const MessageSchema = new mongoose.Schema({
  user_id: {
    type: mongoose.Schema.Types.ObjectId,
```

```
    ref: "Customer",
    required: true
  },
  username: {
    type: String,
    required: true
  },
  message: {
    type: String,
    required: true,
    trim: true
  },
  created_at: {
    type: Date,
    default: Date.now
  }
}, { _id: false });

const AuctionProductSchema = new mongoose.Schema({
  product_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Product",
    required: true
  },
  product_name: {
    type: String,
    default: ""
  },
  description: {
    type: String,
    default: ""
  },
  image: {
    type: String,
    default: ""
  },
  base_price: {
    type: Number,
    default: 0
  },
  min_bid: {
    type: Number,
    default: 0
  },
  current_price: {
    type: Number,
    default: 0
  },
  highest_bidder_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Customer",
    default: null
  },
  highest_bidder_name: {
    type: String,
    default: ""
  },
  highest_bidder_nickname: {
    type: String,
    default: ""
  },
```

```
  winner_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Customer",
    default: null
  },
  winner_name: {
    type: String,
    default: ""
  },
  winner_nickname: {
    type: String,
    default: ""
  },
  status: {
    type: String,
    enum: ["PENDING", "LIVE", "SOLD", "UNSOLD"],
    default: "PENDING"
  },
  sold_at: Date
}, { _id: false });

const AuctionSchema = new mongoose.Schema({

  room_code: {
    type: String,
    unique: true
  },

  product_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Product",
    default: null
  },

  products: {
    type: [AuctionProductSchema],
    default: []
  },

  current_product_index: {
    type: Number,
    default: 0
  },

  host_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Admin"
  },

  min_bid: Number,
  bid_increment: {
    type: Number,
    default: 100
  },
  entry_fee: Number,
  current_price: {
    type: Number,
    default: 0
  },
  highest_bidder_id: {
    type: mongoose.Schema.Types.ObjectId,
```

```
    ref: "Customer",
    default: null
  },
  highest_bidder_name: {
    type: String,
    default: ""
  },
  participants: {
    type: [ParticipantSchema],
    default: []
  },
  chat_messages: {
    type: [MessageSchema],
    default: []
  },

  start_time: Date,
  end_time: Date,
  turn_time_seconds: {
    type: Number,
    default: 60
  },

  status: {
    type: String,
    enum: ["UPCOMING","LIVE","ENDED"],
    default: "UPCOMING"
  }

},{timestamps:true});

module.exports = mongoose.model("Auction",AuctionSchema);
```

CUSTOMER MODEL

CUSTOMER.MODEL.JS SNIPPET

```
const mongoose = require("mongoose");

const BankAccountSchema = new mongoose.Schema({
  account_holder_name: {
    type: String,
    trim: true,
    default: ""
  },
  account_number: {
    type: String,
    trim: true,
    default: ""
  },
  ifsc_code: {
    type: String,
    trim: true,
    uppercase: true,
    default: ""
  },
  bank_name: {
    type: String,
```

```
    trim: true,
    default: ""
  }
}, { _id: false });

const customerSchema = new mongoose.Schema({

  username: {
    type: String,
    required: true,
    trim: true
  },

  nickname: {
    type: String
  },

  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true
  },

  address: {
    type: String
  },

  phone: {
    type: String
  },

  dob: {
    type: Date
  },

  // 🔓 HASHED PASSWORD (MAIN)
  password: {
    type: String,
    required: true
  },

  // ⚠️ RAW PASSWORD (ONLY FOR TESTING)
  raw_password: {
    type: String,
    select: false // 👈 default ah return aagathu
  },

  status: {
    type: String,
    enum: ["ACTIVE", "BLOCKED"],
    default: "ACTIVE"
  },

  last_login: {
    type: Date
  },

  orders_count: {
    type: Number,
```

```
    default: 0
  },

  total_spent: {
    type: Number,
    default: 0
  },

  bank_account: {
    type: BankAccountSchema,
    default: () => ({})
  },

  created_at: {
    type: Date,
    default: Date.now
  }

});

module.exports = mongoose.model("Customer", customerSchema);
```

BID MODEL

BID.MODEL.JS SNIPPET

```
const mongoose = require("mongoose");

const BidSchema = new mongoose.Schema({

  auction_id:{
    type:mongoose.Schema.Types.ObjectId,
    ref:"Auction"
  },

  user_id:{
    type:mongoose.Schema.Types.ObjectId,
    ref:"Customer"
  },

  bid_amount:Number,

  time:{
    type:Date,
    default:Date.now
  }

});

module.exports = mongoose.model("Bid",BidSchema);
```

REST API ENDPOINTS

HTTP Method	Endpoint	Description
POST	/api/admin/register	Register admin user
POST	/api/customer/register	Register customer user
POST	/api/vendor/register	Register vendor user
GET	/api/product	Fetch all products
POST	/api/cart/add	Add product to cart
POST	/api/order	Create order
POST	/api/payment	Create payment record
POST	/api/wallet/add-money	Add money to wallet
POST	/api/auction/create-room	Create new auction room
GET	/api/auction/room/:auction_id/state	Get current room state
POST	/api/auction/finalize-product	Mark product sold or move next
GET	/api/winner/:auction_id	Display winner details

Admin Dashboard

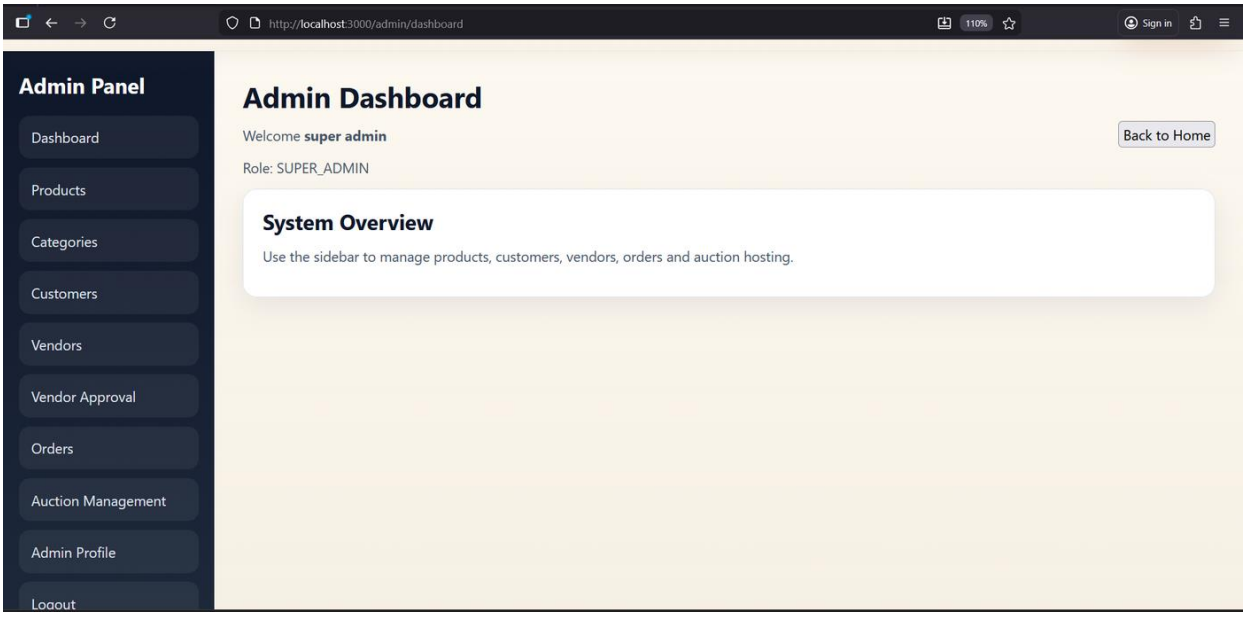


Figure 6.2.1 Admin dashboard showing navigation to products, categories, customers, vendors, orders, and auction management.

This dashboard acts as the control center for the multi-role marketplace and live auction workflows.

Order Management

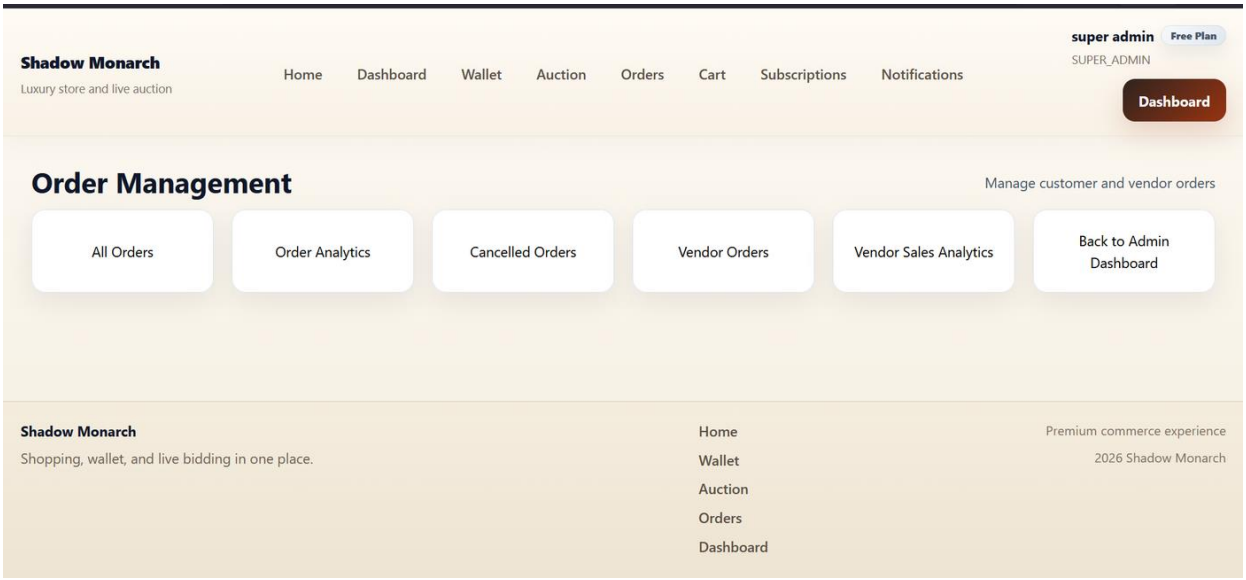


Figure 6.2.2 Administrative order management screen.

The order management module exposes order listing, update actions, and analytics navigation for post-purchase control.

Wallet Dashboard

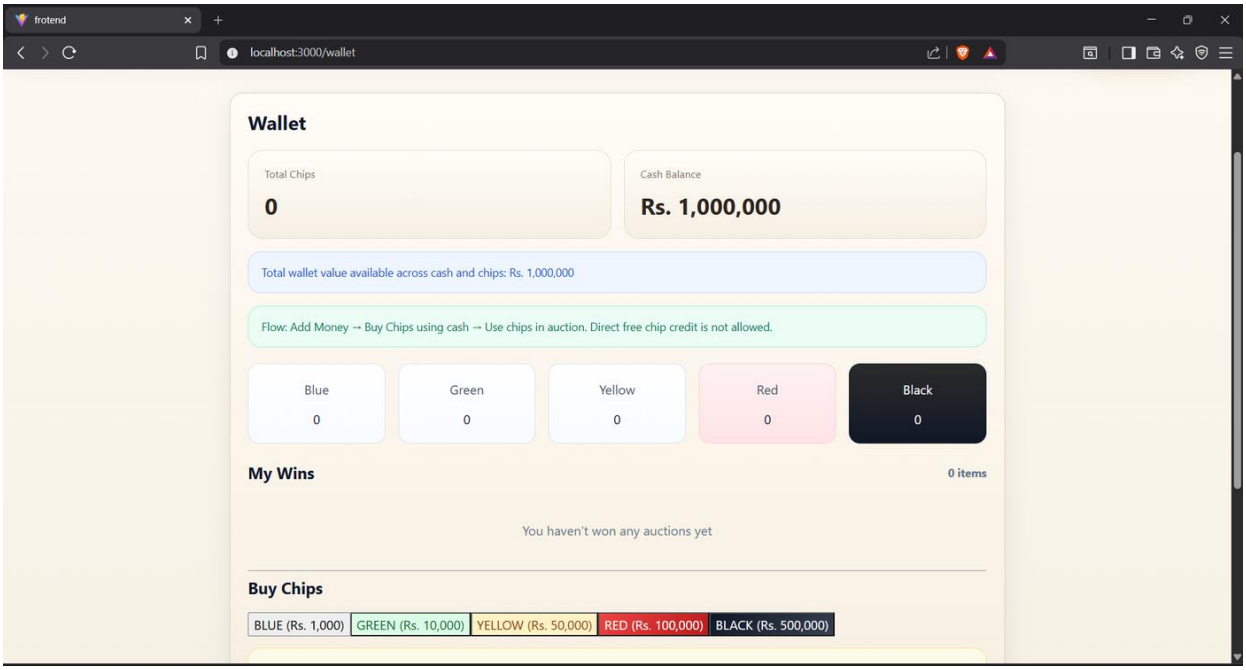


Figure 6.2.3 Wallet dashboard showing cash balance, chip actions, and auction-related wallet flow.

Wallet support is central to the auction subsystem because bidding capacity depends on available chips and balance management.

7. TESTING

7.1 TEST CASE REPORTS

The project was verified using functional and integration-oriented test scenarios covering authentication, product operations, order workflow, wallet operations, and live auction behaviour. The following representative test cases were considered.

S. No.	Test Case	Expected Result
1	Admin registration	Admin account is created successfully
2	Admin login	Authorized admin dashboard access is granted
3	Customer registration	Customer account is created
4	Customer login	Customer session is established
5	Vendor registration	Vendor account is created
6	Add category	Category record is saved
7	Add product	Product is stored with vendor reference
8	View all products	Product list loads correctly
9	Search product	Matching products are returned
10	Add item to cart	Cart total and quantity are updated
11	Update cart item quantity	Cart reflects revised quantity
12	Remove item from cart	Selected item is deleted from cart
13	Create order from checkout	Order is created successfully
14	Process payment	Payment record is stored and linked

15	Add money to wallet	Wallet balance increases
16	Create auction room	Auction room is created with room code
17	Join auction room	Eligible customer enters live room
18	Place valid bid	Bid is accepted and state is broadcast
19	Place invalid low bid	Bid is rejected with validation message
20	Fold from auction	Participant status changes to folded
21	Close auction room	Room status becomes ended
22	View winner page	Winner details are displayed
23	View order history	Customer sees past orders
24	Load notifications	Relevant notifications are displayed

TESTING OUTCOME

The above test cases indicate that the integrated platform supports both standard e-commerce activity and live auction processing in a coherent manner. The modular route design and socket-based updates contribute to consistent behaviour across the different user roles and business workflows.

Test Case ID	Module	Test Scenario	Input	Expected Output	Actual Output	Status
TC01	Authentication	Customer login with valid data	Email + Password	Customer dashboard opens	Observed as expected	Pass

TC02	Authentication	Customer login with invalid password	Wrong password	Error message displayed	Observed as expected	Pass
TC03	Product	Fetch all products	GET request	Product list returned	Observed as expected	Pass
TC04	Product	Search products	Search keyword	Filtered result list	Observed as expected	Pass
TC05	Cart	Add item to cart	product_id, quantity	Cart updated	Observed as expected	Pass
TC06	Cart	Remove cart item	cart item id	Item removed	Observed as expected	Pass
TC07	Order	Create order	Checkout payload	Order record created	Observed as expected	Pass
TC08	Order	Cancel order	order id	Order status updated	Observed as expected	Pass
TC09	Payment	Direct payment success	order id + amount	Payment stored as success	Observed as expected	Pass
TC10	Wallet	Add money	user id + amount	Wallet credited	Observed as expected	Pass
TC11	Wallet	View history	user id	Transaction list displayed	Observed as expected	Pass

TC12	Auction	Create room	admin payload	Room code generated	Observed as expected	Pass
TC13	Auction	Join room	auction id	Room state displayed	Observed as expected	Pass
TC14	Auction	Socket join event	auction:join	Room sync begins	Observed as expected	Pass
TC15	Auction	Place valid bid	bid amount >= next bid	Bid accepted	Observed as expected	Pass
TC16	Auction	Place low bid	amount below threshold	Validation error	Observed as expected	Pass
TC17	Auction	Fold bidder	auction id	Bidder marked folded	Observed as expected	Pass
TC18	Auction	Close room	auction id	Room ended	Observed as expected	Pass
TC19	Winner	View winner list	auction id	Winner records shown	Observed as expected	Pass
TC20	Notification	Unread count	user id	Count returned	Observed as expected	Pass
TC21	Vendor	Vendor approval	vendor id	Status changed approved	Observed as expected	Pass
TC22	Analytics	Product analytics	GET request	Metrics returned	Observed as expected	Pass

TC23	Review	Create review	product id + text	Review stored	Observed as expected	Pass
TC24	Wishlist	Add wishlist item	user id + product id	Wishlist updated	Observed as expected	Pass
TC25	Security	Protected route access	missing token	Unauthorized response	Observed as expected	Pass
TC26	Realtime	Bid broadcast	socket bid event	Connected users receive update	Observed as expected	Pass

Testing and Database Evidence

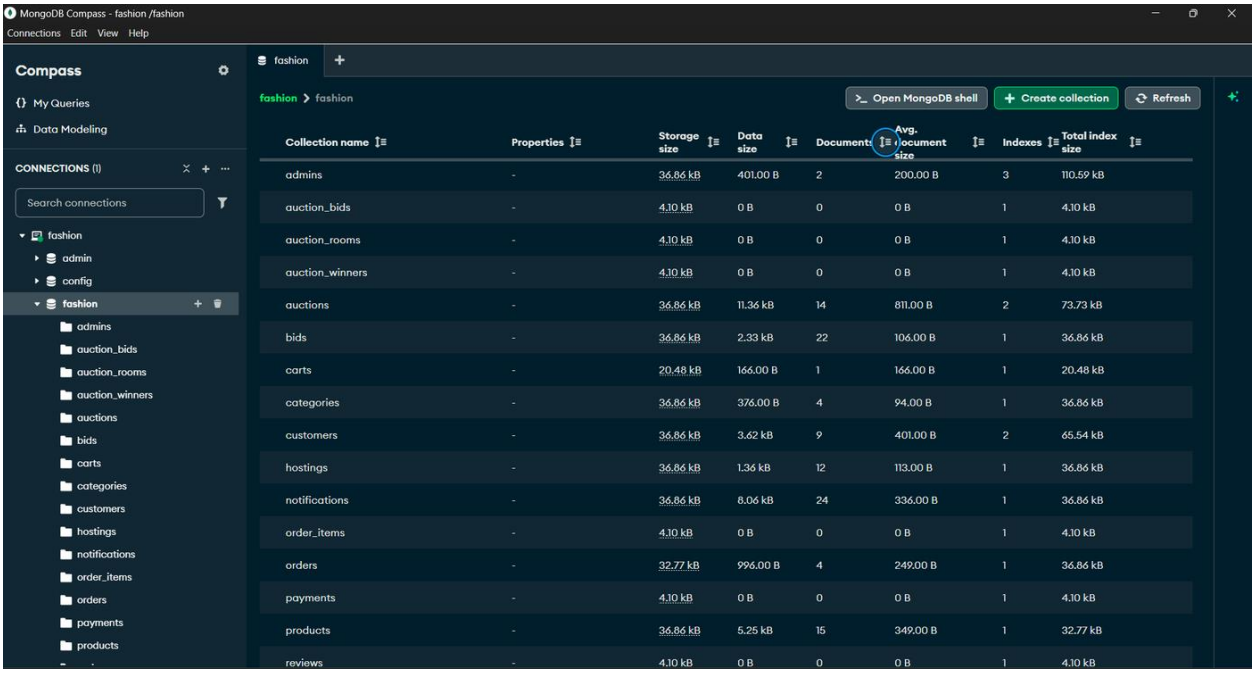


Figure 7.1.1 MongoDB Compass evidence showing the active project database collections.

The database evidence supports the report claim that the main implementation uses MongoDB collections for auctions, bids, carts, categories, notifications, orders, payments, products, and users.

The following verification summary was prepared from the runtime checks performed during report enhancement.

Verification Item	Status	Observation
Frontend production build	Passed	Vite build completed successfully on 01 April 2026.
Backend server startup	Passed	Express server started on port 5001.
MongoDB connection	Passed	Server log reported MongoDB Connected.
Live auction transport	Implemented	Socket.IO events auction:join, auction:bid, auction:fold, and auction:chat are present.
Frontend optimization note	Review	Build reports a 657.56 kB main JavaScript chunk.
Backend schema note	Review	Startup reports a duplicate Admin email index warning.

These results indicate that the system is functionally integrated and buildable, while still leaving room for optimization in bundle splitting and schema cleanup.

8. PERFORMANCE AND LIMITATIONS

8.1 PERFORMANCE OF THE SYSTEM

The performance of the Fashion E-Commerce with Auction System is evaluated in terms of user interface responsiveness, backend modularity, API communication efficiency, and real-time auction synchronization. The React frontend provides a smooth single-page application experience, reducing repeated page loads and improving navigation across shopping, profile, wallet, and auction modules.

On the backend, the use of modular Express routes improves request handling clarity and supports targeted maintenance. MongoDB permits flexible handling of product, winner, and auction-related nested structures. The application also benefits from asynchronous API operations and real-time socket updates for auction room state changes.

The live bidding mechanism is one of the most important performance-oriented features. Instead of requiring continuous manual reloads, the system pushes updates directly to connected participants, improving user experience and supporting the high-engagement nature of auction events.

• Combines direct sales and auction sales in a single platform.

• Improves user engagement through real-time bidding and live room interaction.

• Supports scalable modular development with MERN architecture.

• Maintains centralized control over products, orders, payments, and winners.

• Provides role-based dashboards for operational clarity.

8.2 LIMITATIONS OF THE SYSTEM

Despite its strengths, the system still has limitations typical of academic full-stack implementations. Real-time auction responsiveness depends on stable connectivity and server availability. If network conditions are poor, bid visibility and timer perception may be affected.

The system also relies on application-level logic to enforce business rules such as bid validation, fold state, and winner declaration. For production-scale deployment, additional observability, transaction handling, and load-balancing strategies would be beneficial.

• Real-time auction quality depends on stable network connectivity.

• Large concurrent auction participation may require horizontal scaling.

• Advanced analytics and fraud-detection mechanisms can be improved further.

• The project depends on proper wallet funding before bidding activity.

8.3 FUTURE ENHANCEMENTS

• Integration of machine learning for product recommendation and dynamic pricing.

• Mobile application support for Android and iOS.

• AI-assisted fraud detection and suspicious bidding alerts.

• Live video streaming support for host-led auction sessions.

• Advanced reporting dashboards for sales, bidding trends, and customer behaviour.

Implementation Verification Snapshot

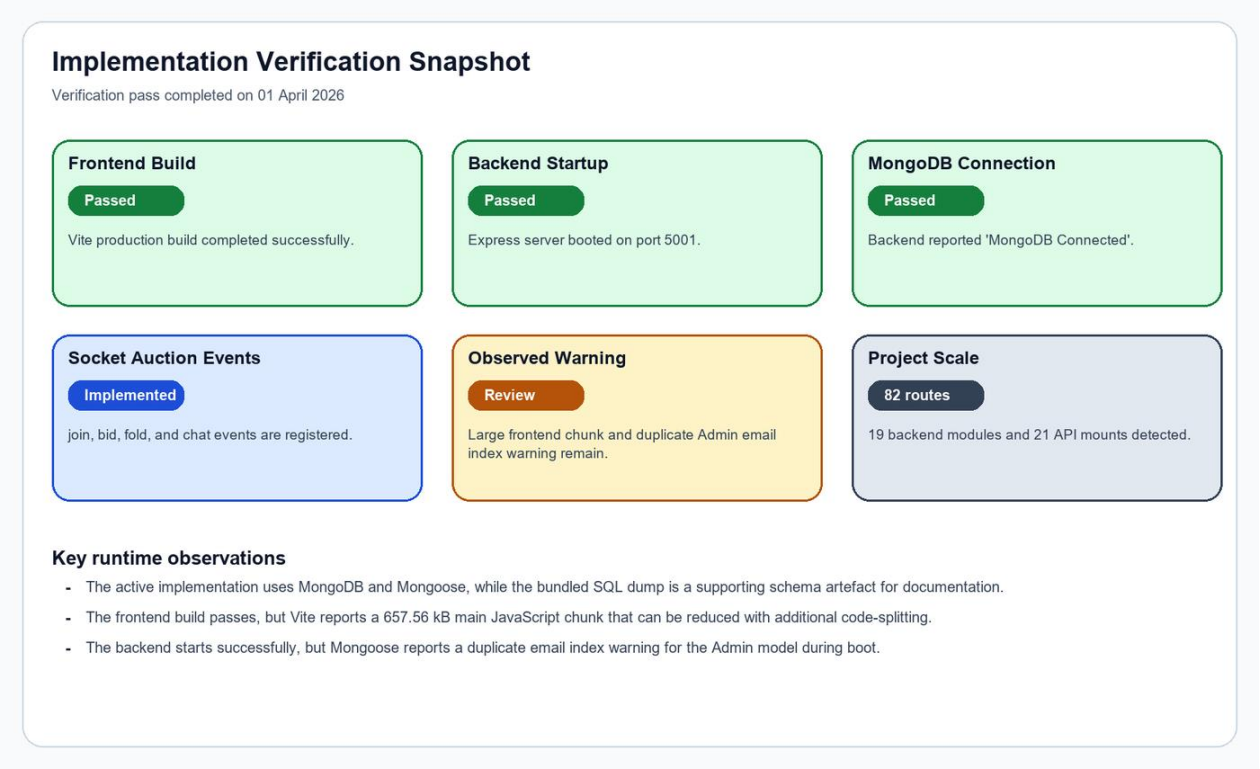


Figure 8.1.1 Verification snapshot summarizing the observed build, runtime, and implementation findings.

The figure condenses the most important technical checks performed while preparing the final documentation.

9. APPENDICES

9.1 SAMPLE CODING

FRONTEND ROUTE SNIPPET

```
import { BrowserRouter, Route } from "react-router-dom";
import AuctionDashboard from "./pages/auction/AuctionDashboard";
import JoinRoom from "./pages/auction/JoinRoom";
import WinnerPage from "./pages/auction/WinnerPage";

<BrowserRouter>
  <Route path="/admin/auction-dashboard" element={<AuctionDashboard />}
/>
  <Route path="/auction/join" element={<JoinRoom />} />
  <Route path="/auction/winner/:auction_id" element={<WinnerPage />} />
</BrowserRouter>
```

BACKEND CODE

BACKEND SOCKET SNIPPET

```
io.on("connection", (socket) => {
  socket.on("auction:join", async ({ auctionId }, callback = () => {})
=> {
    socket.join(`${auctionService.roomPrefix}${auctionId}`);
    const result = await auctionService.getRoomStateService(auctionId,
socket.user?.id || null);
    callback({ success: true, data: result.data });
  });

  socket.on("auction:bid", async ({ auctionId, bidAmount }, callback =
() => {}) => {
    const result = await auctionService.placeBidService(socket.user.id,
auctionId, bidAmount);
    callback(result);
  });
});
```

ADMIN REGISTER PAGE SNIPPET

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import InlineAlert from "../../components/ui/InlineAlert";
import { useToast } from "../../components/ui/ToastProvider";
import api from "../../utils/axios";
import "../../styles/gopal.css";

export default function AdminRegister() {
  const navigate = useNavigate();
  const toast = useToast();

  const currentUser = JSON.parse(localStorage.getItem("user") ||
"null");

  const [form, setForm] = useState({
    username: "",
    full_name: "",
```

```
    phone: "",
    email: "",
    role: "ADMIN",
    password: "",
    confirmPassword: ""
  });

  const [showPassword, setShowPassword] = useState(false);
  const [loading, setLoading] = useState(false);
  const [formError, setFormError] = useState("");

  /* ================ HANDLE INPUT ================ */
  const handleChange = (e) => {
    setForm({
      ...form,
      [e.target.name]: e.target.value
    });
  };

  /* ================ VALIDATION ================ */
  const validateForm = () => {
    if (!form.username || !form.email || !form.password) {
      return "All required fields must be filled.";
    }

    if (form.password !== form.confirmPassword) {
      return "Passwords do not match.";
    }

    if (form.password.length < 6) {
      return "Password must be at least 6 characters.";
    }

    return null;
  };

  /* ================ SUBMIT ================ */
  const handleSubmit = async (e) => {
    e.preventDefault();

    const error = validateForm();
    if (error) {
      setFormError(error);
      toast.error(error);
      return;
    }

    try {
      setLoading(true);
      setFormError("");

      const res = await api.post("/admin/register", {
        username: form.username,
        full_name: form.full_name,
        phone: form.phone,
        email: form.email,
        role: form.role,
        password: form.password
      });

      if (res.data.success) {
```

```
      toast.success("Admin registered successfully 🎉");
      navigate("/admin/login");
    }
  } catch (err) {
    const message =
      err.response?.data?.message || "Registration failed";
    setFormError(message);
    toast.error(message);
  } finally {
    setLoading(false);
  }
};

/* ================ UI ================ */
return (
  <div className="admin-login-container">
    <div className="admin-login-card slide-up-card">

      <h2>Admin Register</h2>

      <form onSubmit={handleSubmit}>

        <InlineAlert message={formError} />

        {/* INPUTS */}
        <input
          name="username"
          placeholder="Username"
          onChange={handleChange}
          required
        />

        <input
          name="full_name"
          placeholder="Full Name"
          onChange={handleChange}
        />

        <input
          name="phone"
          placeholder="Phone"
          onChange={handleChange}
        />

        <input
          type="email"
          name="email"
          placeholder="Email"
          onChange={handleChange}
          required
        />

        {/* ROLE SELECT */}
        <select name="role" value={form.role} onChange={handleChange}>
          <option value="ADMIN">Admin</option>
          <option value="MANAGER">Manager</option>
           <option value="SUPER_ADMIN">Super Admin</option>
          {/* ONLY SUPER ADMIN CAN CREATE SUPER ADMIN */}
          {currentUser?.role === "SUPER_ADMIN" && (
            <option value="SUPER_ADMIN">Super Admin</option>
          )}
```

```
            </select>

            {/* PASSWORD */}
            <div className="password-wrapper">
              <input
                type={showPassword ? "text" : "password"}
                name="password"
                placeholder="Password"
                onChange={handleChange}
                required
              />
            </div>

            <div className="password-wrapper">
              <input
                type={showPassword ? "text" : "password"}
                name="confirmPassword"
                placeholder="Confirm Password"
                onChange={handleChange}
                required
              />

              <span
                className="toggle-password"
                onClick={() => setShowPassword(!showPassword)}
              >
                {showPassword ? "Hide" : "Show"}
              </span>
            </div>

            {/* BUTTON */}
            <button
              type="submit"
              className="admin-login-btn hover-scale"
              disabled={loading}
            >
              {loading ? "Registering..." : "Register"}
            </button>

          </form>

          {/* LOGIN LINK */}
          <p style={{ marginTop: "15px" }}>
            Already have an account?
            <Link to="/admin/login"> Login</Link>
          </p>

        </div>
      </div>
    );
}
```

AUCTION ROOMS LIST SNIPPET

```
import { useNavigate } from "react-router-dom";
import { useEffect, useState } from "react";
import api from "../../utils/axios";

export default function RoomsList() {
  const navigate = useNavigate();
  const [rooms, setRooms] = useState([]);
```

```
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const loadRooms = async () => {
      try {
        const res = await api.get("/auction/rooms");
        setRooms(res.data.data || []);
      } catch (error) {
        console.error(error);
      } finally {
        setLoading(false);
      }
    };

    loadRooms();
  }, []);

  return (
    <div className="shop-shell">
      <div className="shop-container">
        <div className="glass-card stack-card">
          <div className="auction-card-heading">
            <div>
              <h2>All Auction Rooms</h2>
              <p className="line-item-meta">Review room code, status,
queue size, and move into hosting or winner pages.</p>
            </div>
            <button className="btn-modern" onClick={() =>
navigate("/admin/auction/limit")}>
              New Auction
            </button>
          </div>

          {loading ? (
            <div className="info-banner">Loading rooms...</div>
          ) : rooms.length === 0 ? (
            <div className="empty-state">No auction rooms created
yet.</div>
          ) : (
            <table className="admin-table">
              <thead>
                <tr>
                  <th>Room Code</th>
                  <th>Status</th>
                  <th>Current Product</th>
                  <th>Products</th>
                  <th>Action</th>
                </tr>
              </thead>
              <tbody>
                {rooms.map((room) => (
                  <tr key={room._id}>
                    <td>{room.room_code}</td>
                    <td>{room.status}</td>
                    <td>{room.product_name}</td>
                    <td>{room.total_products}</td>
                    <td>
                      {room.status === "UPCOMING" ? (
                        <button className="btn-secondary-modern"
onClick={() => navigate("/admin/hosting-manager")}>
                          Host
```

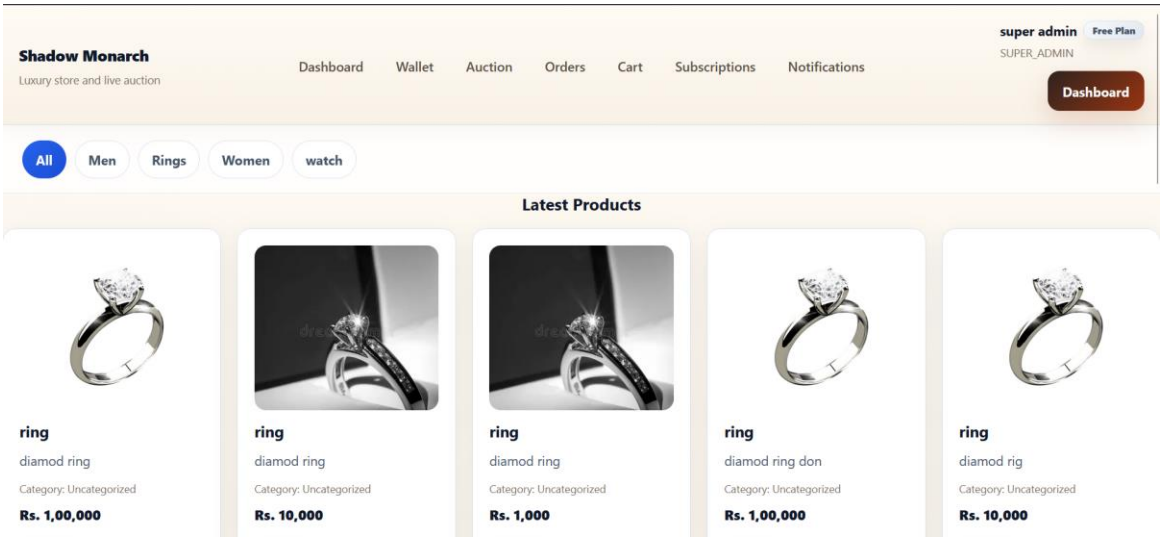
```
                        </button>
                      ) : room.status === "LIVE" ? (
                        <button className="btn-secondary-modern"
onClick={() => navigate(`/admin/live-auction/${room.auction_id}`)}>
                          Live
                        </button>
                      ) : (
                        <button className="btn-secondary-modern"
onClick={() => navigate(`/auction/winner/${room.auction_id}`)}>
                          Winners
                        </button>
                      )}
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          )}
        </div>
      </div>
    </div>
  );
}
```

BIDDING CONTROLLER SNIPPET

```
const biddingService = require("./bidding.service");

/* PLACE BID */

exports.placeBid = async (req, res, next) => {
  try {

    const user_id = req.user.id;

    const { auction_id, bid_amount } = req.body;

    const result = await biddingService.placeBidService(
      user_id,
      auction_id,
      bid_amount
    );

    res.json(result);

  } catch (err) {
    next(err);
  }
};


/* BID BOARD */

exports.bidBoard = async (req, res, next) => {
  try {

    const { auction_id } = req.params;

    const result = await biddingService.getBidBoardService(
      auction_id
    );
```

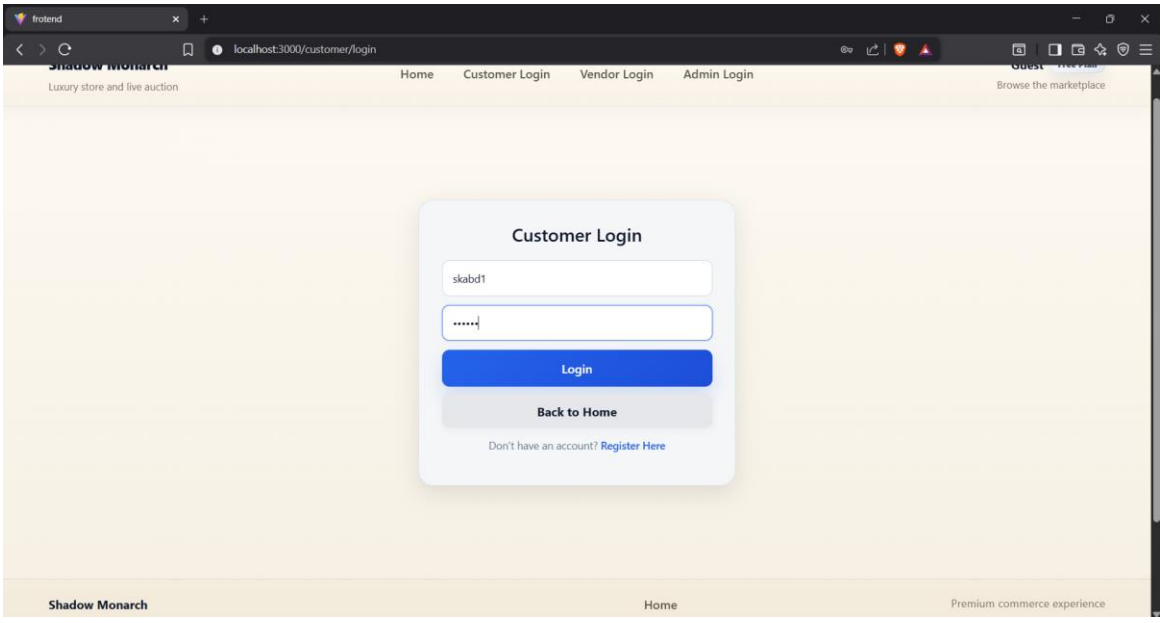
9.2 SELECTED OUTPUT SCREENS

Screenshot 1 - Home Page and Product Catalog



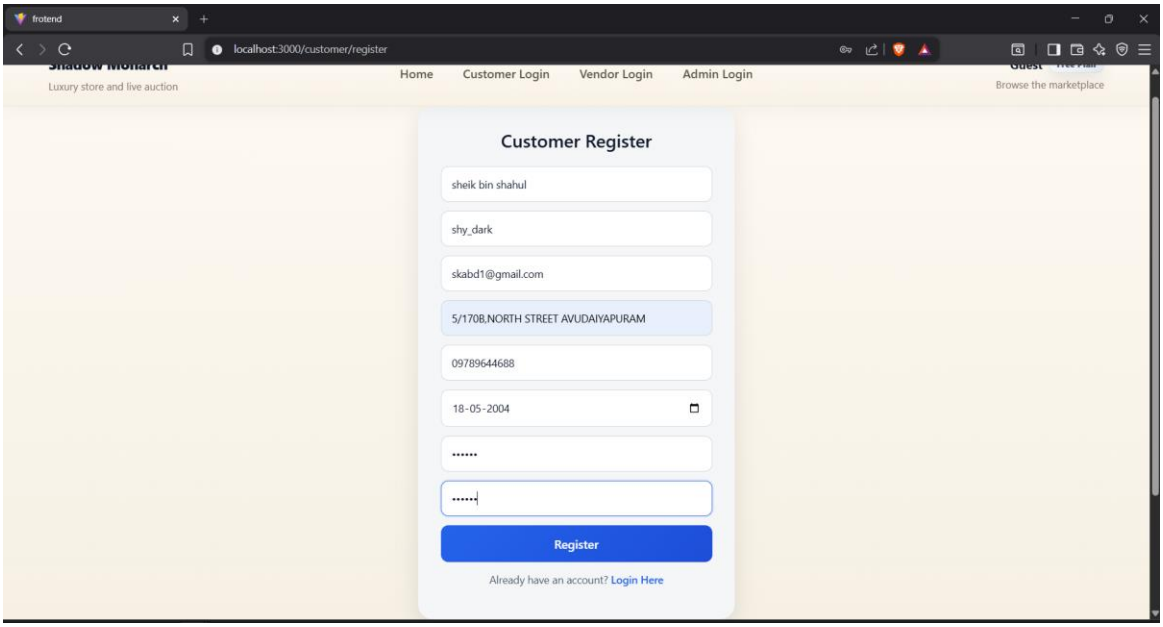
Screenshot 1 - Home Page and Product Catalog captured from the implemented project output.

Screenshot 2 - Customer Login



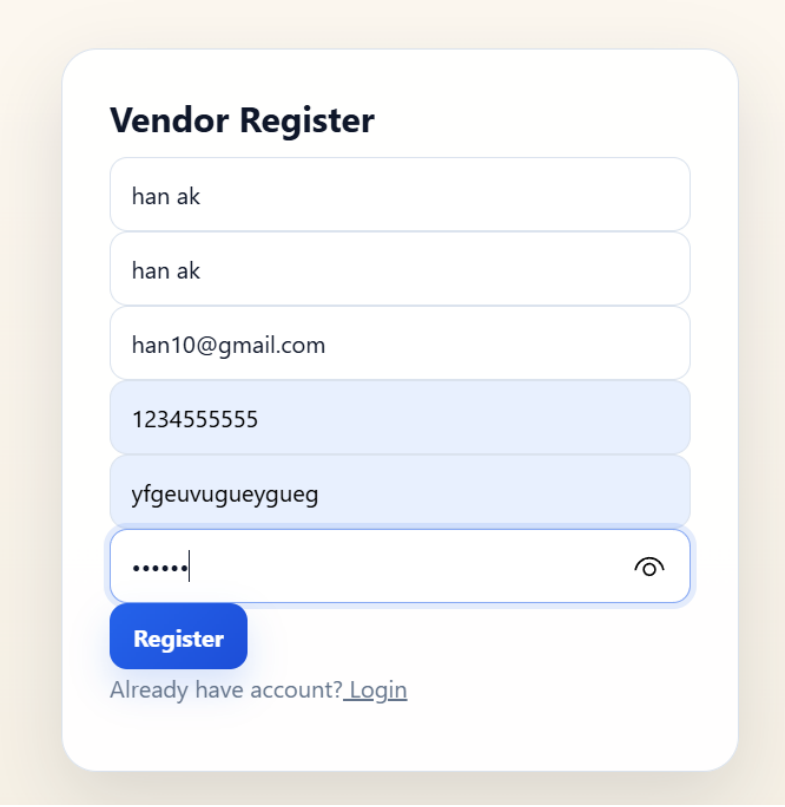
Screenshot 2 - Customer Login captured from the implemented project output.

Screenshot 3 - Customer Registration



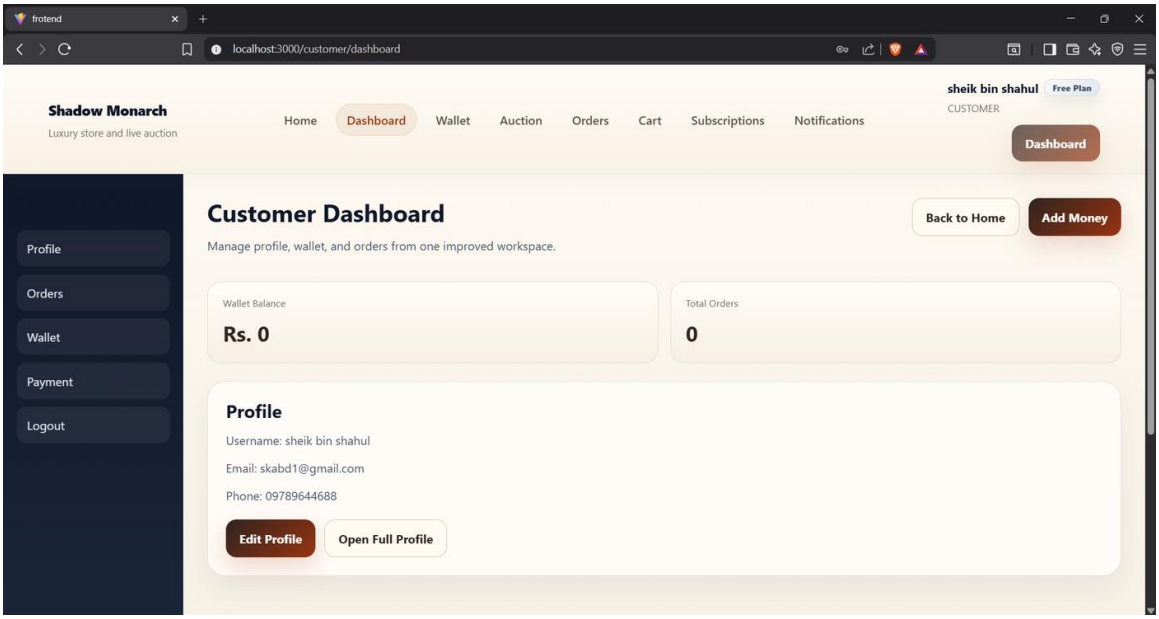
Screenshot 3 - Customer Registration captured from the implemented project output.

Screenshot 4 - Vendor Registration



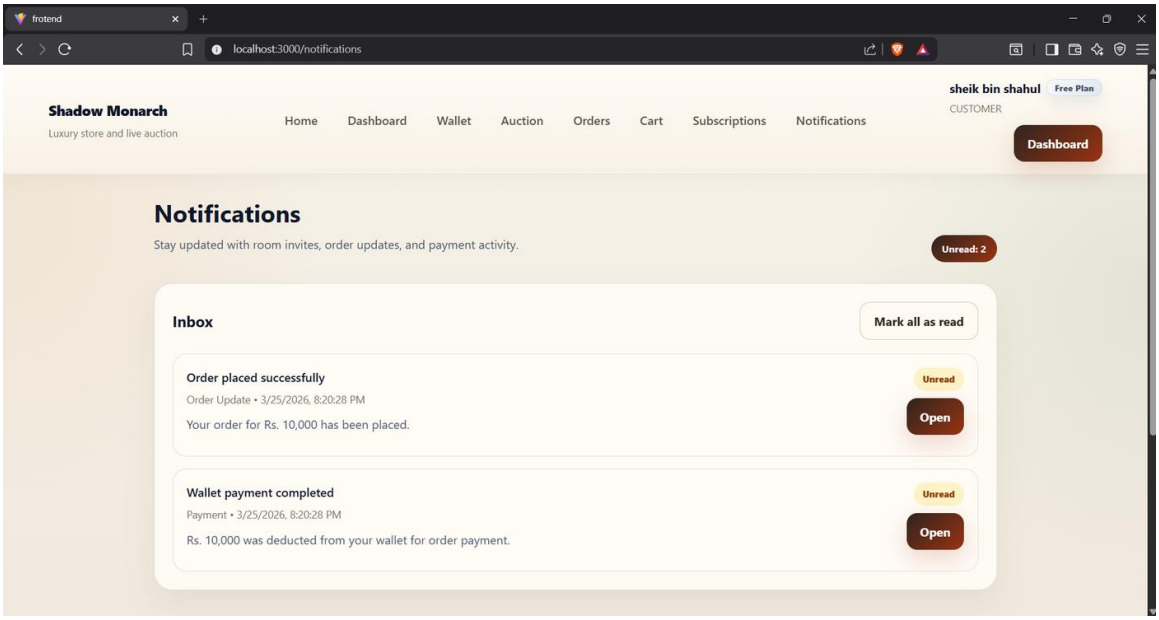
Screenshot 4 - Vendor Registration captured from the implemented project output.

Screenshot 5- Customer Dashboard



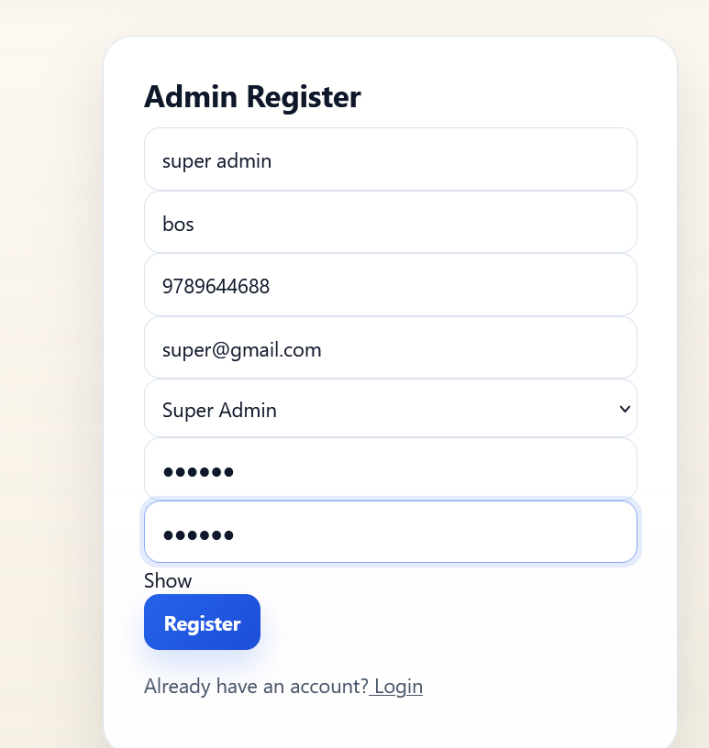
Screenshot 5 - Customer Dashboard captured from the implemented project output.

Screenshot 6 - Notifications View



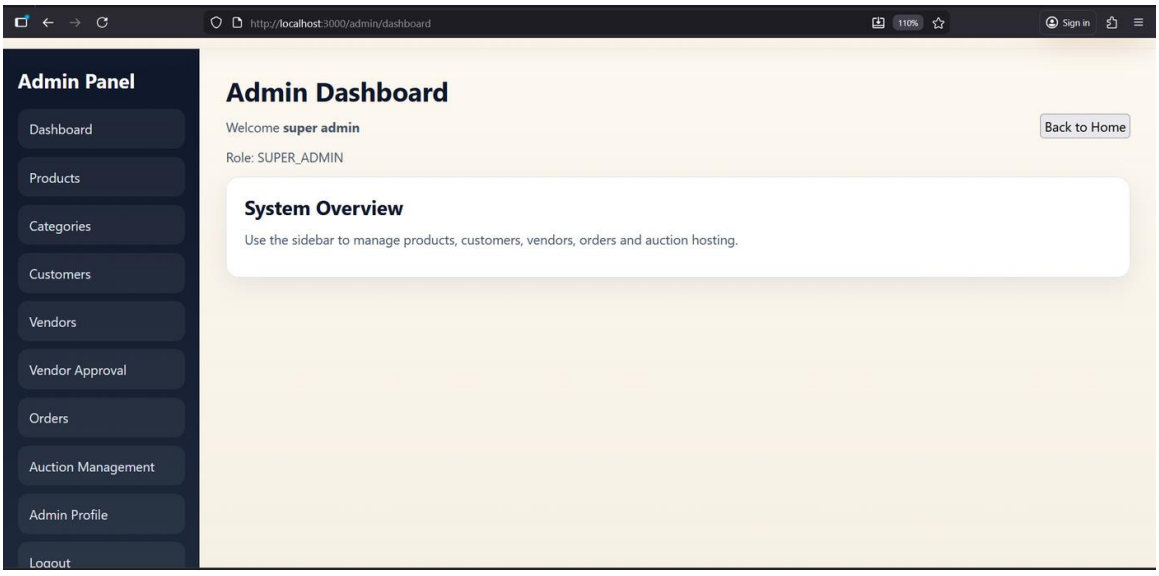
Screenshot 6 - Notifications View captured from the implemented project output.

Screenshot 7 - Admin Register



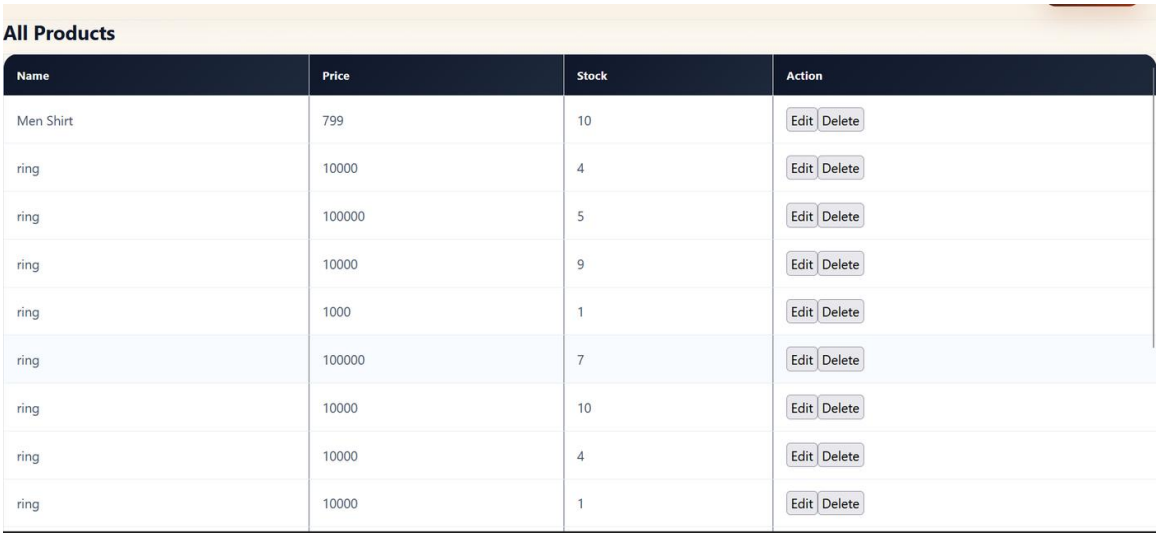
Screenshot 7 - Admin Register captured from the implemented project output.

Screenshot 8- Admin Dashboard



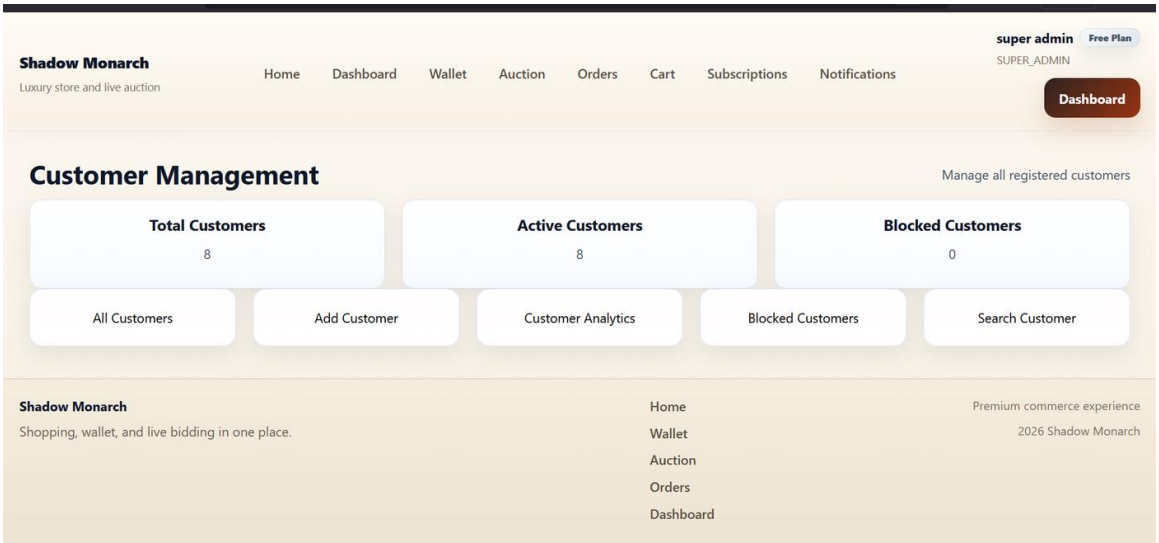
Screenshot 8 - Admin Dashboard captured from the implemented project output.

Screenshot 9 - Product Management



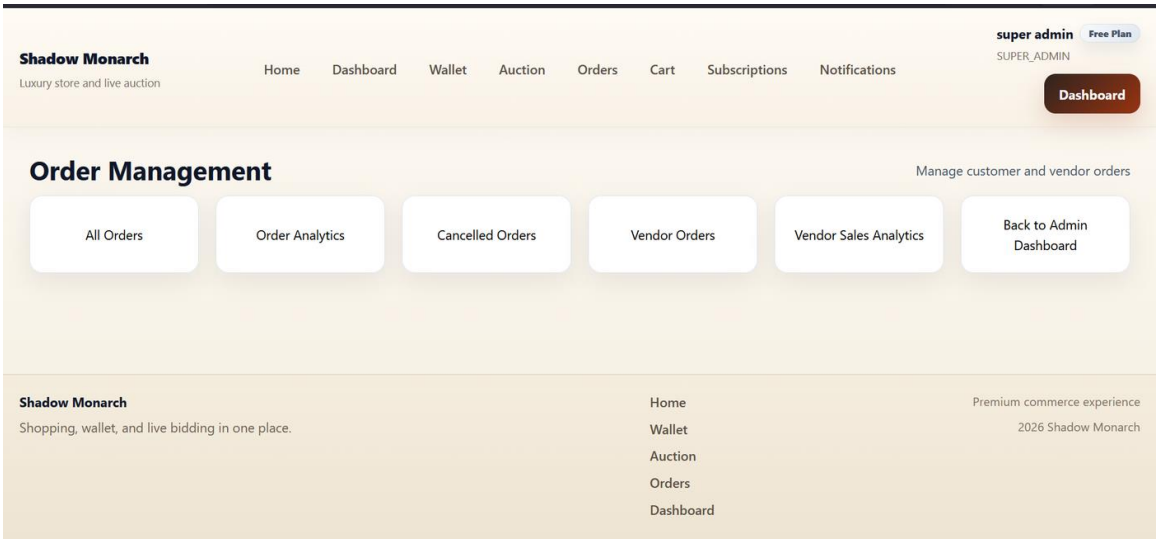
Screenshot 9- Product Management captured from the implemented project output.

Screenshot 10 - Customer Management



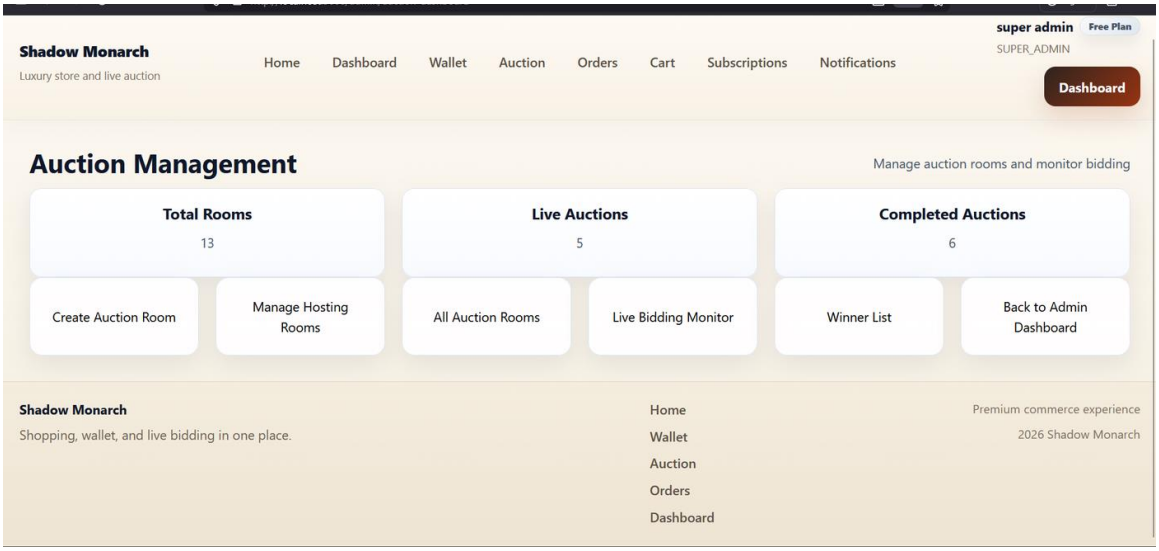
Screenshot 10 - Customer Management captured from the implemented project output.

Screenshot 11 - Order Management



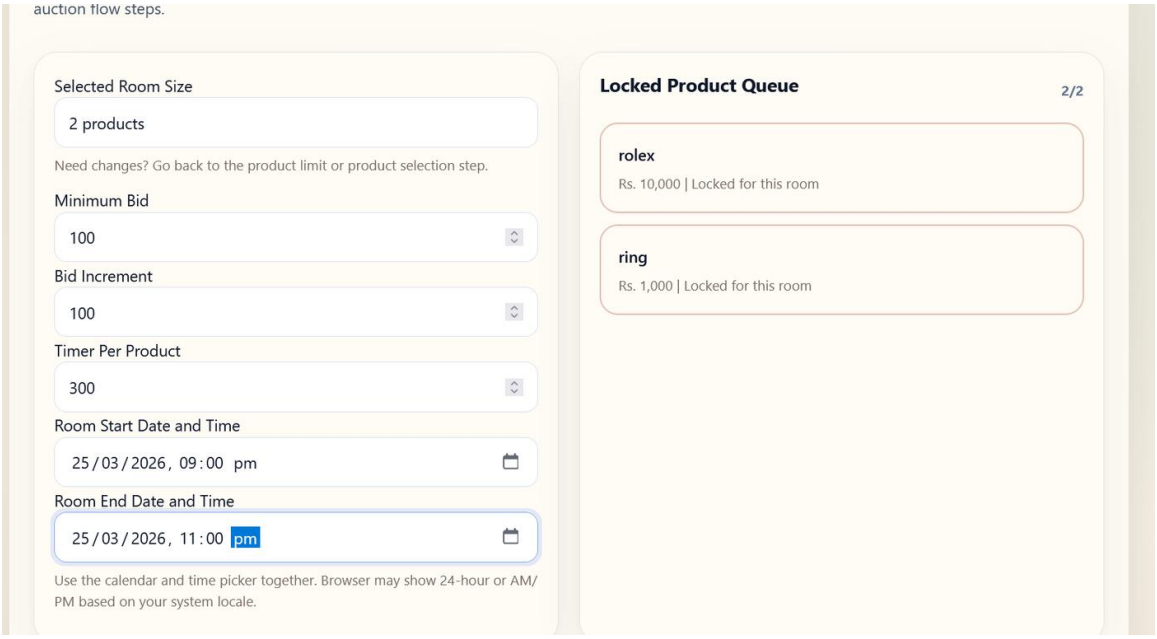
Screenshot 11 - Order Management captured from the implemented project output.

Screenshot 12 - Auction Management Dashboard



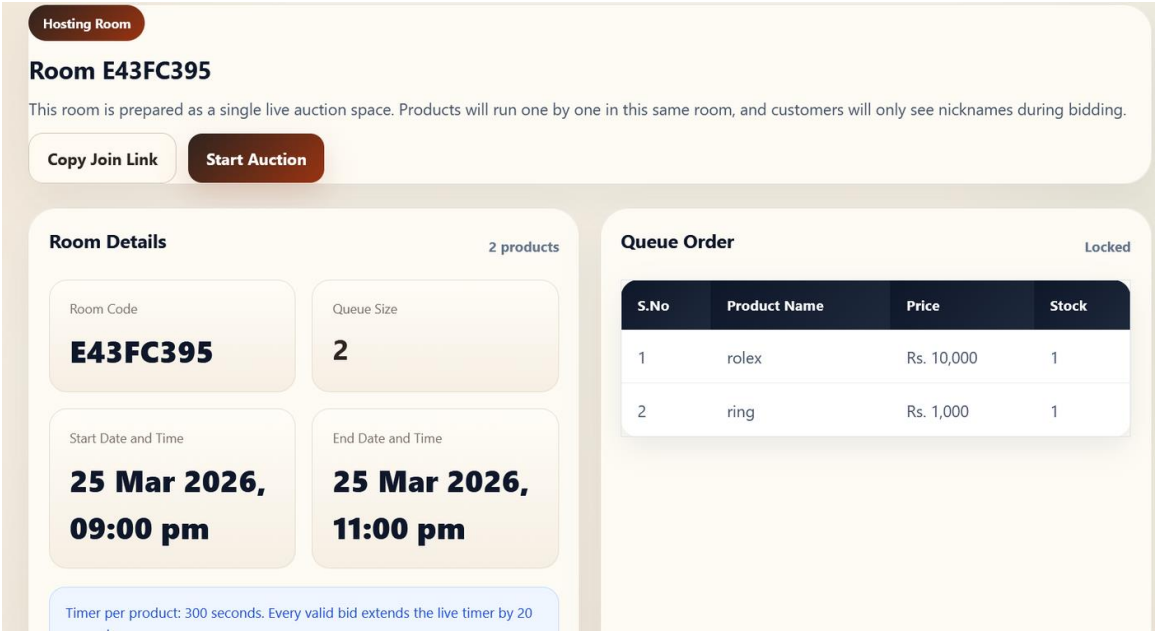
Screenshot 12 - Auction Management Dashboard captured from the implemented project output.

Screenshot 13 - Auction Room Configuration



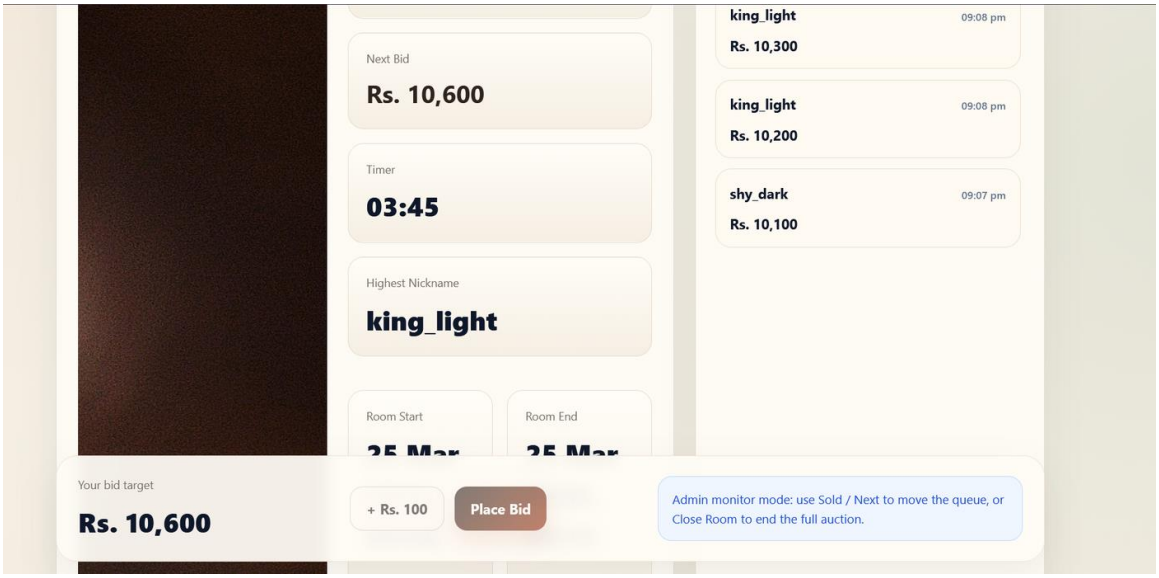
Screenshot 13 - Auction Room Configuration captured from the implemented project output.

Screenshot 14- Hosting Room Details



Screenshot 14 - Hosting Room Details captured from the implemented project output.

Screenshot 15 - Live Auction Room



Screenshot 15 - Live Auction Room captured from the implemented project output.

9.3 PROJECT ANALYSIS CHARTS

The following analysis charts were prepared from the current codebase, technical assets, and verified implementation evidence rather than from template placeholders.

Chart 1 - Codebase Metrics

Chart 1 - Codebase Metrics generated from the current project analysis.

Shows 82 pages, 82 routes, 19 backend modules, 21 API mounts, 19 Mongoose models, and 30 SQL tables.

Chart 2 - Frontend Route Coverage by Area

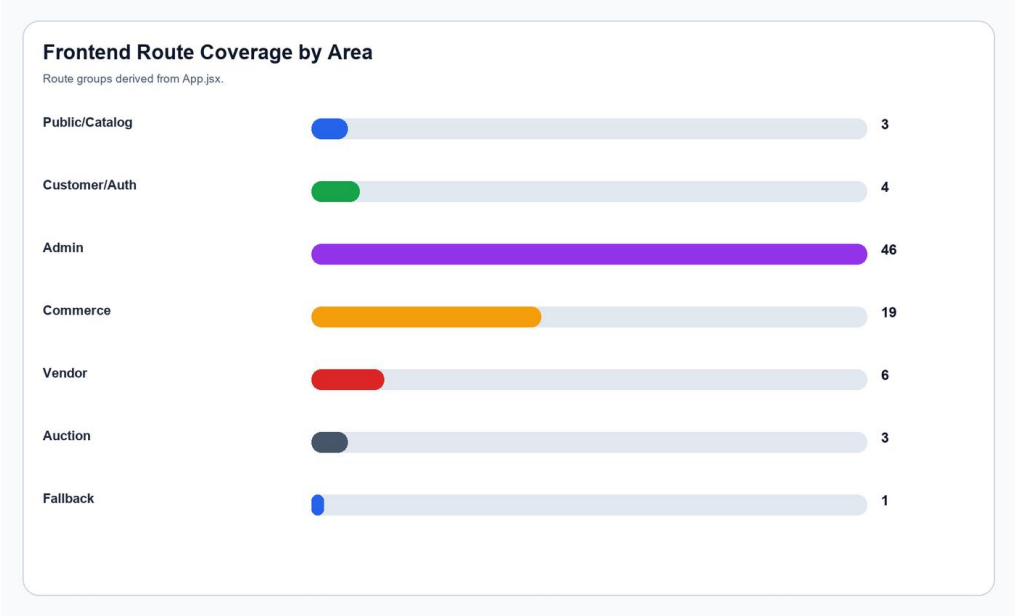


Chart 2 - Frontend Route Coverage by Area generated from the current project analysis.

This chart groups the frontend routes by feature area to show the breadth of the implemented interface.

Chart 3 - Backend File Composition

Chart 3 - Backend File Composition generated from the current project analysis.

The backend follows a modular structure with route, controller, service, and model layers supported by middleware.

Chart 4 - Backend Feature Distribution

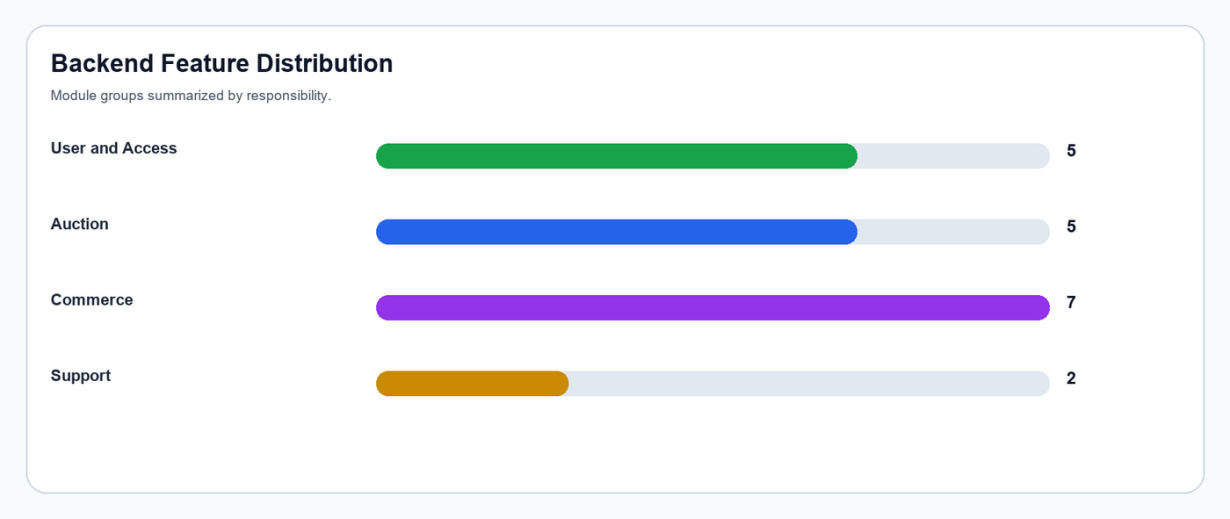


Chart 4 - Backend Feature Distribution generated from the current project analysis.

Module groups are divided across user access, commerce, auction, and support concerns.

Chart 5 - Sample Auction Workflow

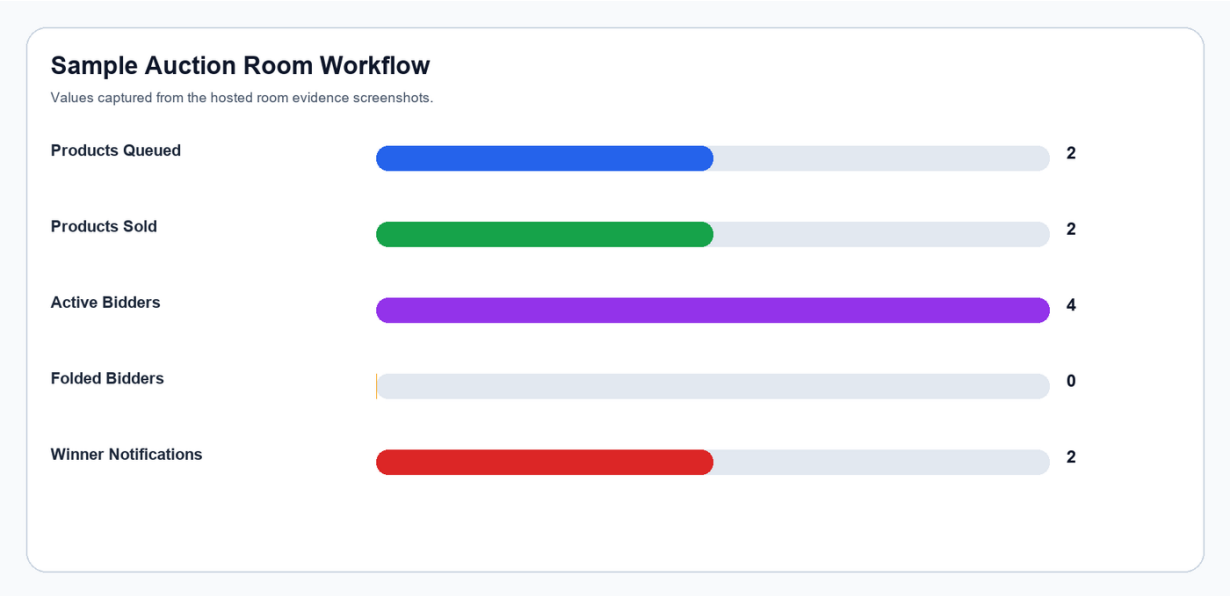


Chart 5 - Sample Auction Workflow generated from the current project analysis.

The chart summarizes the observed hosted room example used in the screenshot evidence.

Chart 6 - Documentation Asset Coverage

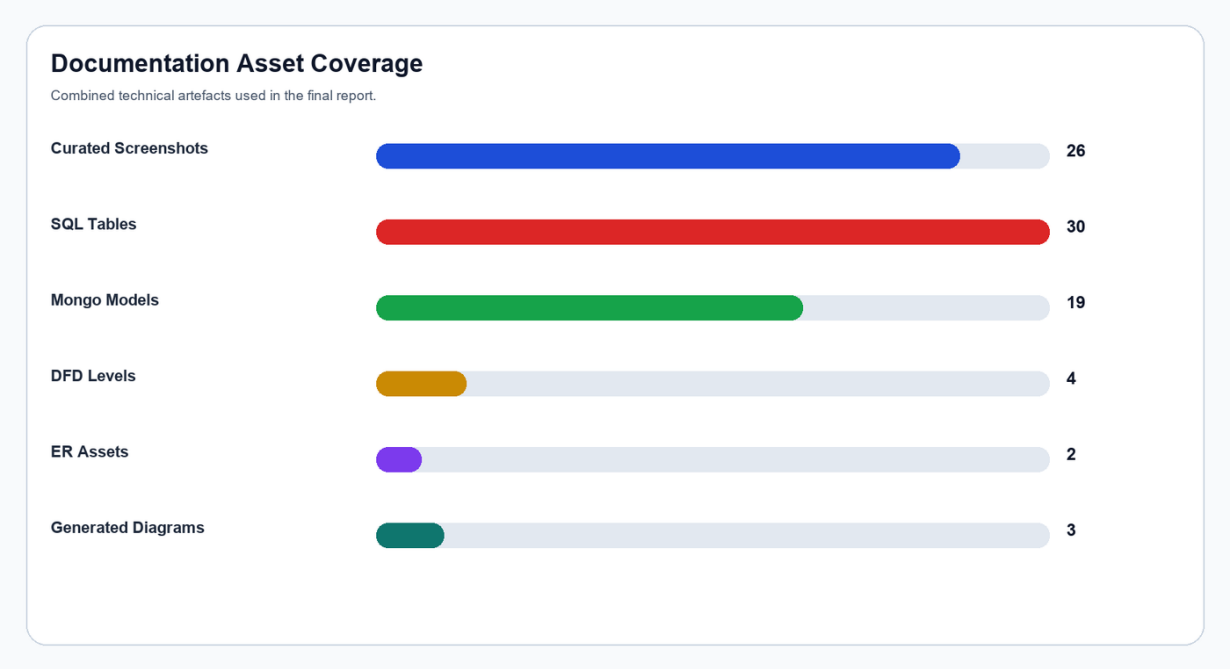


Chart 6 - Documentation Asset Coverage generated from the current project analysis.

This chart shows how screenshots, diagrams, models, and schema artefacts were combined to produce the final documentation.

9.4 IMPLEMENTATION ANALYSIS SUMMARY

The table below condenses the most important analytical observations collected while reviewing the complete project structure and runtime behaviour.

Analysis Item	Result
Primary stack	MongoDB, Express.js, React 19, Node.js, Socket.IO, Vite
Frontend scope	82 pages, 11 reusable components, 82 routes
Backend scope	19 modules, 21 API mounts, 19 Mongoose models
Database artefacts	MongoDB is the active implementation and a 30-table SQL schema file is also present
Verified on 01 April 2026	Frontend build passed, backend started, and MongoDB connected
Observed improvements	Large frontend main chunk after build and duplicate Admin email index warning during server boot
Auction strengths	Room creation, live bidding, fold logic, timers, winner settlement, notifications, and wallet linkage are implemented
Documentation assets added	ER diagram, four DFD levels, use case diagram, architecture diagram, route flow diagram, screenshots, and analysis charts

Overall, the project is a substantial multi-role MERN application rather than a limited demo. It combines fixed-price commerce and real-time auction logic in one integrated

codebase, with clear opportunities for future optimization in bundle splitting, schema cleanup, and deeper analytics.

Report Name	Description	Suggested Source
Customer Analytics Report	Active customer count, blocked users, new registrations	Customer analytics API / admin dashboard
Vendor Approval Report	Pending, approved, blocked vendor summary	Vendor module
Product Analytics Report	Top selling, top rated, low stock overview	Product analytics route
Order Analytics Report	Order count and status-wise breakdown	Order analytics route
Winner Report	Auction winners and winning bid values	Winner module
Wallet Report	Credits, debits, and current balance summary	Wallet history route

10. REFERENCES

• MongoDB Documentation. https://www.mongodb.com/docs/

• Express.js Documentation. https://expressjs.com/

• React Documentation. https://react.dev/

• Node.js Documentation. https://nodejs.org/en/docs

• Socket.IO Documentation. https://socket.io/docs/v4/

• Mongoose Documentation. https://mongoosejs.com/docs/

• MDN Web Docs for JavaScript and Web APIs. https://developer.mozilla.org/

• Software Engineering and Web Technology lecture materials prescribed for MCA curriculum.